# *Digital Communication IC Design*

# *Lecture-7: Adaptive Signal Processing and Channel Equalization*

***Chih-Feng Wu (吳志峰), PhD.***

***Department of Electronics Engineering, Chang Gung University, Taoyuan, Taiwan (ROC)***

***May 6, 2026***

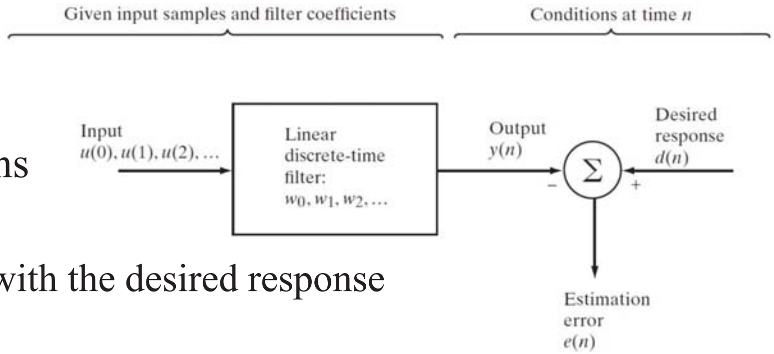


## *Outline*

- Wiener Filter (Refer to S. Haykin, 2013)
- Method of Steepest Descent (Refer to S. Haykin, 2013)
- Least-Mean-Square Algorithm (Refer to S. Haykin, 2013)
- System Model of Single-Carrier Baseband Transmission
- Linear Equalizer and LMS Algorithm
- Decision-Directed Linear Equalization
- Decision Feedback Equalization (DFE)

# Wiener Filter

- Block diagram representation of the statistical filter problem




  - Two major functions
    - Filtering
    - Error estimation with the desired response
  - Two restrictions
    - Linear for the filter
    - **Discrete-time** for operating the filter, which means that the filter can be realized by digital hardware and software
  - Two choices for the filter specification
    - ***Finite*** duration of the filter
    - ***Statistical criterion*** used for the system **optimization**

S. Haykin, 2013



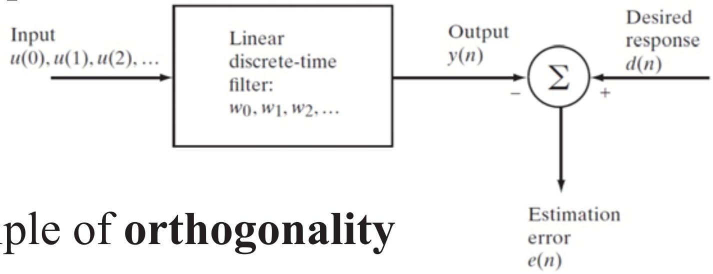
# Wiener Filter -- Statistical Criteria

- Statistical criteria
  - Optimize the filter by minimizing a ***cost function*** (**performance index**)
- Three types of the cost function
  - 1. Mean-square value of the estimation value (MSE)
  - 2. Expectation of the absolute value of the estimation value
  - 3. Expectation of third or higher powers of the absolute value of the estimation value
- Obvious advantage of option-1 (MSE) over the other two
  - A **second-order** dependence for the cost function on the unknown coefficients in the coefficients (impulse response) of the filter

S. Haykin, 2013

# *Wiener Filter – Mathematical Solution*

- Two approaches for the statistical optimization problem
  - The principle of **orthogonality**
  - The **error-performance surface** that describes the second-order dependence of the cost function on the filter coefficients



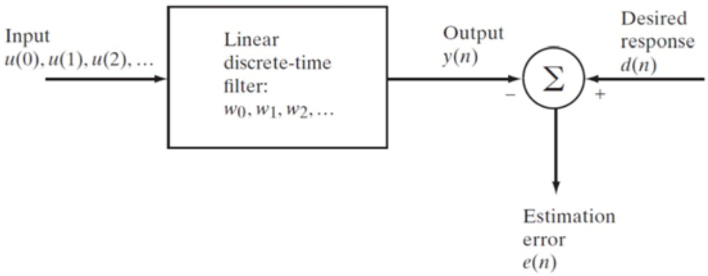

- The principle of **orthogonality**
  - The filter output: $y(n) = \sum_{k=0} w_k^* u(n-k), \qquad n = 0, 1, 2, \ldots,$
  - The error estimation: $e(n) = d(n) - y(n)$
  - The cost function of MSE: $J = \mathbb{E}[e(n)e^*(n)]$
    - Minimize the MSE of $e(n)$ $= \mathbb{E}[|e(n)|^2],$

S. Haykin, 2013



# *Wiener Filter -- The Principle of Orthogonality*

- The principle of **orthogonality**
  - The filter output: $y(n) = \sum_{k=0}^{\infty} w_k^* u(n-k), \qquad n = 0, 1, 2, \ldots,$
  - The error estimation: $e(n) = d(n) - y(n)$
  - The cost function of MSE: $J = \mathbb{E}[e(n)e^*(n)]$
    - Minimize the MSE of $e(n)$ $= \mathbb{E}[|e(n)|^2],$
    - Determine the operating condition for attaining the minimum of $J$




  - In general, all of $u(n)$, $d(n)$, $y(n)$, $e(n)$ and $w_k(n)$ $(k = 0, 1, 2, \ldots)$ are complex-value.

S. Haykin, 2013

## *Wiener Filter -- The Principle of Orthogonality (cont'd)*

- Find the gradient of $J$ to reach the minimum of $J$
  - Define the gradient vector $\nabla_k J$ respecting for the coefficients (k=0, 1, 2, …)

    $$\nabla_k J = \frac{\partial}{\partial w_k} J = \frac{\partial}{\partial a_k} J + j \frac{\partial}{\partial b_k} J, \text{ where } w_k = a_k + j b_k$$

  - The gradient vector $\nabla_k J = 0$ for reaching the minimum of $J$
    - The filter is said to be optimum in the MSE sense.

$$\nabla_k J = \mathbb{E}\left[\frac{\partial e(n)}{\partial a_k} e^*(n) + \frac{\partial e^*(n)}{\partial a_k} e(n) + \frac{\partial e(n)}{\partial b_k} je^*(n) + \frac{\partial e^*(n)}{\partial b_k} je(n)\right]$$

$$\nabla_k J = -2\mathbb{E}[u(n-k)e^*(n)]$$

S. Haykin, 2013



## *Wiener Filter -- The Principle of Orthogonality (cont'd)*

- The optimum condition when the estimation error $e_o$ reaching the minimum value

$$\mathbb{E}[u(n-k)e_o^*(n)] = 0, \qquad k = 0, 1, 2, \ldots.$$

  - The necessary and sufficient condition for the **cost function J** to attain **its minimum value** is that the corresponding value of the estimation error $e_o$ is **orthogonal** to **each input sample**.
- Corollary to the principle of orthogonality
  - Exam the correlation between $y(n)$ and $e(n)$

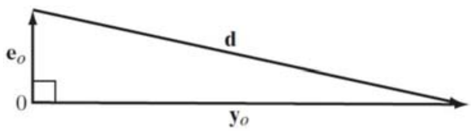
$$\mathbb{E}[y(n)e^*(n)] = \mathbb{E}\left[\sum_{k=0}^{\infty} w_k^* u(n-k)e^*(n)\right]$$

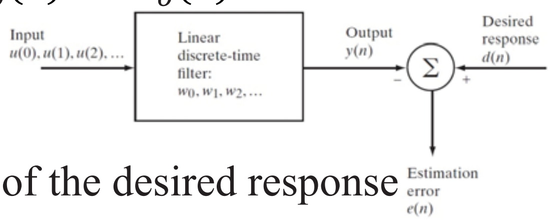
$$= \sum_{k=0}^{\infty} w_k^* \mathbb{E}[u(n-k)e^*(n)].$$

S. Haykin, 2013

# Wiener Filter -- The Principle of Orthogonality (cont'd)

- Corollary to the principle of orthogonality (cont'd)
  - Let $y_o$ denote the output produced by the filter optimized in MSE sense, when $e_o$ denoting the corresponding estimation error.
    $\mathbb{E}[y_o(n)e_o^*(n)] = 0$
    - When the filter operates in it optimum condition, the estimation of the desired response defined by the filter output $y_o(n)$ and the corresponding estimation error $e_o(n)$ are orthogonal to each other.
  - Geometric interpretation
    - The relationship between $d(n)$, $y_o(n)$ and $e_o(n)$




  - $\hat{d}(n|\mathcal{U}_n) = y_o(n)$: the estimation of the desired response that is optimized in the MSE sense

S. Haykin, 2013



# Wiener-Hopf Equations

- According the principle of orthogonality, we have

$$\mathbb{E}[u(n-k)e_o^*(n)] = 0, \qquad k = 0, 1, 2, \ldots.$$

$$\mathbb{E}\left[u(n-k)\left(d^*(n) - \sum_{i=0}^{\infty} w_{oi}u^*(n-i)\right)\right] = 0, \qquad k = 0, 1, 2, \ldots$$

  - $w_{oi}$: the $i$th coefficient in the impulse response of the optimal filter
- Re-arrange the above equation as

$$\sum_{i=0}^{\infty} w_{oi}\mathbb{E}[u(n-k)u^*(n-i)] = \mathbb{E}[u(n-k)d^*(n)], \qquad k = 0, 1, 2\ldots$$

  - $E[u(n-k)u^*(n-i)]$: defined as the **autocorrelation** function of the filter input for a lag of $(i-k)$, i.e., $r\ (i-k) = E[u(n-k)u^*(n-i)]$

# Wiener-Hopf Equations (cont'd)

- Re-arrange the above equation as (cont'd)
  - $E[u(n-k)d^*(n)]$: defined as the cross-correlation between the filter input $u(n-k)$ and the desired response $d(n)$ for a lag of $(0-k)$, i.e., $p(-k) = E[u(n-k)d^*(n)]$
- Wiener-Hopf equations
  - According $r(i-k)$ and $p(-k)$, the system equation can be revised as

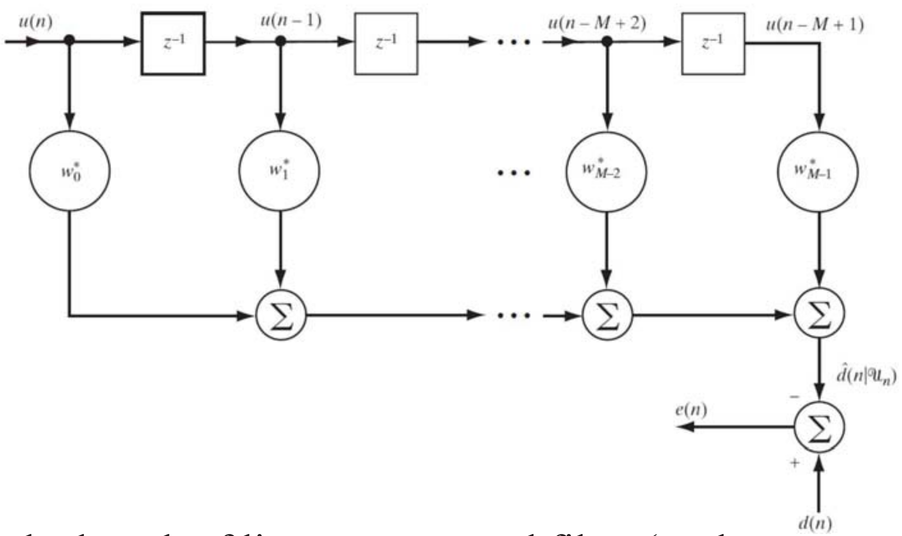
$$\sum_{i=0}^{\infty} w_{oi} r(i-k) = p(-k), \qquad k = 0, 1, 2, \ldots$$

  - The system equation defines the optimum filter coefficients is terms of two correlation functions such as $r(i-k)$ and $p(-k)$



# Wiener-Hopf Equations (cont'd)

- Solution of the Wiener-Hopf equations for linear transversal filter




  - $M$: the length of linear transversal filter (or the tap number)
  - Three basic operators: storage, multiplication and addition

# *Wiener-Hopf Equations (cont'd)*

- Matrix formulation of the Wiener-Hopf equations
  - Autocorrelation matrix (M×M) **R**

$$\mathbf{R} = \mathbb{E}[\mathbf{u}(n)\mathbf{u}^{\mathrm{H}}(n)]$$

$$\mathbf{u}(n) = [u(n), u(n-1), \ldots, u(n-M+1)]^{\mathrm{T}}$$

$$\mathbf{R} = \begin{bmatrix} r(0) & r(1) & \cdots & r(M-1) \\ r^*(1) & r(0) & \cdots & r(M-2) \\ \vdots & \vdots & \ddots & \vdots \\ r^*(M-1) & r^*(M-2) & \cdots & r(0) \end{bmatrix}$$

  - Cross-correlation matrix (M×1) **p**

$$\mathbf{p} = \mathbb{E}[\mathbf{u}(n)d^*(n)]$$

$$\mathbf{p} = [p(0), p(-1), \ldots, p(1-M)]^{\mathrm{T}}$$



# *Wiener-Hopf Equations (cont'd)*

- Matrix formulation of the Wiener-Hopf equations
  - The compact form of the Wiener-Hopf equations

$$\mathbf{R}\mathbf{w}_o = \mathbf{p}$$

  - Optimum weight vector $\mathbf{w}_o$: $\mathbf{w}_o = [\mathbf{w}_{o0}, \mathbf{w}_{o1}, \ldots, \mathbf{w}_{o,M-1}]^{\mathrm{T}}$

- Finally, the optimum weight vector can be found as

$$\mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$$

  - **R** is **nonsigular**. ($\because$ $\mathbf{R}^{-1}$ exists.)
  - The computation of the optimum tap-weight vector requires knowledge of two quantities.
    - **R**
    - **P**

# Wiener Filter – Error-Performance Surface

- Cost function $J$ derived according to $e(n) = d(n) - \sum_{k=0}^{M-1} w_k^* u(n-k)$

$$J = \mathbb{E}[e(n)e^*(n)]$$

$$= \mathbb{E}[|d(n)|^2] - \sum_{k=0}^{M-1} w_k^* \mathbb{E}[u(n-k)d^*(n)] - \sum_{k=0}^{M-1} w_k \mathbb{E}[u^*(n-k)d(n)]$$

$$+ \sum_{k=0}^{M-1}\sum_{i=0}^{M-1} w_k^* w_i \mathbb{E}[u(n-k)u^*(n-i)].$$

  - Variance of the desired response: $\sigma_d^2 = \mathbb{E}[|d(n)|^2]$
  - Cross-correlation: $p(-k) = \mathbb{E}[u(n-k)d^*(n)]$
    $p^*(-k) = \mathbb{E}[u^*(n-k)d(n)]$
  - Auto-correlation: $r(i-k) = \mathbb{E}[u(n-k)u^*(n-i)]$

$$J = \sigma_d^2 - \sum_{k=0}^{M-1} w_k^* p(-k) - \sum_{k=0}^{M-1} w_k p^*(-k) + \sum_{k=0}^{M-1}\sum_{i=0}^{M-1} w_k^* w_i r(i-k)$$



# Wiener Filter – Error-Performance Surface (cont’d)

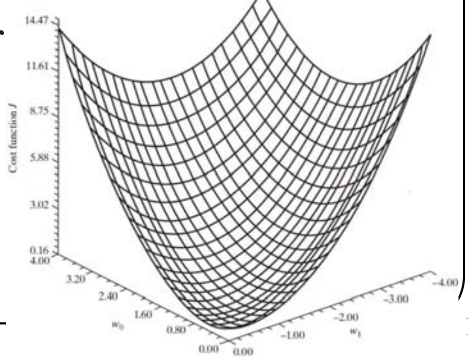
- Summary

$$J = \sigma_d^2 - \sum_{k=0}^{M-1} w_k^* p(-k) - \sum_{k=0}^{M-1} w_k p^*(-k) + \sum_{k=0}^{M-1}\sum_{i=0}^{M-1} w_k^* w_i r(i-k)$$

  - A second-order function of the tap weights in the filter
  - The dependence of J on the tap weights $w_0, w_1, \dots, w_{M-1}$ as bowl-shaped $(M+1)$-dimensional surface with $M$ degrees of freedom represented by the tap weights of the filter
  - $(M+1)$-dimensional composed of $\sigma_d^2$ and $M$ tap weights
- Example: J on $M = 2$ tap-weight vector

$$J(w_0, w_1) = 0.9486 - 2[0.5272, -0.4458]\begin{bmatrix} w_0 \\ w_1 \end{bmatrix} + [w_0, w_1]\begin{bmatrix} 1.1 & 0.5 \\ 0.5 & 1.1 \end{bmatrix}\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

$$= 0.9486 - 1.0544\, w_0 + 0.8961\, w_1 + w_0 w_1 + 1.1(w_0^2 + w_1^2).$$



## *Wiener Filter – Minimum Mean-Square Error*

- Minimum MSE (MMSE)
  - $\hat{d}(n|\mathcal{U}_n) = y_o(n)$: the estimation of the desired response $d(n)$

$$\hat{d}(n|\mathcal{U}_n) = \sum_{k=0}^{M-1} w_{ok}^* u(n-k) = \mathbf{w}_o^{\mathrm{H}}\mathbf{u}(n),$$

    - $\mathbf{u}(n)$: the tap-input vector with zero-mean
    - $\mathbf{w_o}(n)$: the optimum tap-weight vector
  - Therefore, $\hat{d}(n|\mathcal{U}_n)$ is zero-mean.
  - The variance of $\hat{d}(n|\mathcal{U}_n)$

$$\sigma_{\hat{d}}^2 = \mathbb{E}[\mathbf{w}_o^{\mathrm{H}}\mathbf{u}(n)\mathbf{u}^{\mathrm{H}}(n)\mathbf{w}_o] = \mathbf{w}_o^{\mathrm{H}}\mathbb{E}[\mathbf{u}(n)\mathbf{u}^{\mathrm{H}}(n)]\mathbf{w}_o = \mathbf{w}_o^{\mathrm{H}}\mathbf{R}\mathbf{w}_o,$$

$$\sigma_{\hat{d}}^2 = \mathbf{p}^{\mathrm{H}}\mathbf{w}_o = \mathbf{p}^{\mathrm{H}}\mathbf{R}^{-1}\mathbf{p}$$

  - MMSE of Wiener filter

$$J_{min} = \sigma_d^2 - \sigma_{\hat{d}}^2 = \sigma_d^2 - \mathbf{p}^{\mathrm{H}}\mathbf{w}_o = \sigma_d^2 - \mathbf{p}^H\mathbf{R}^{-1}\mathbf{p}$$



## *Canonical Form of the Error-Performance Surface*

- Matrix form of MSE

$$J(\mathbf{w}) = \sigma_d^2 - \mathbf{w}^{\mathrm{H}}\mathbf{p} - \mathbf{p}^{\mathrm{H}}\mathbf{w} + \mathbf{w}^{\mathrm{H}}\mathbf{R}\mathbf{w}$$

  - **R** is **nonsingular**. ($\because$ $\mathbf{R^{-1}}$ exists.)

$$J(\mathbf{w}) = \sigma_d^2 - \mathbf{p}^{\mathrm{H}}\mathbf{R}^{-1}\mathbf{p} + (\mathbf{w} - \mathbf{R}^{-1}\mathbf{p})^{\mathrm{H}}\mathbf{R}(\mathbf{w} - \mathbf{R}^{-1}\mathbf{p})$$

  - Considering the min J with the optimum tap weights vector $\mathbf{w_o}$

$$\min_{\mathbf{w}} J(\mathbf{w}) = \sigma_d^2 - \mathbf{p}^{\mathrm{H}}\mathbf{R}^{-1}\mathbf{p} \quad \text{and} \quad \mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$$

  - MSE

$$J(\mathbf{w}) = J_{\min} + (\mathbf{w} - \mathbf{w}_o)^{\mathrm{H}}\mathbf{R}(\mathbf{w} - \mathbf{w}_o)$$

    - Depend on the minimizing tap weights vector $\mathbf{w_o}$

# Canonical Form of the Error-Performance Surface (cont'd)

- Consideration of eigen-decomposition for the correlation matrix $\mathbf{R}$

  $\mathbf{R} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^{\mathrm{H}}$

  - $\boldsymbol{\Lambda}$: the eigenvalues $\lambda_0, \lambda_1, \ldots, \lambda_{M-1}$ of $\mathbf{R}$
  - $\mathbf{Q}$: the matrix with its columns eigenvectors $\mathbf{q}_0, \mathbf{q}_1, \ldots, \mathbf{q}_{M-1}$ associated with these eigenvalues

  $J = J_{\min} + (\mathbf{w} - \mathbf{w}_o)^{\mathrm{H}}\mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^{\mathrm{H}}(\mathbf{w} - \mathbf{w}_o)$

  - Define a transformed version $\mathbf{v}$ of the difference between $\mathbf{w}$ and $\mathbf{w}_o$

  $\mathbf{v} = \mathbf{Q}^{\mathrm{H}}(\mathbf{w}_o - \mathbf{w})$

- Canonical form of $J$

  $J = J_{\min} + \mathbf{v}^{\mathrm{H}}\boldsymbol{\Lambda}\mathbf{v}$



# Canonical Form of the Error-Performance Surface (cont'd)

- Canonical form of $J$ (cont'd)

$$J = J_{\min} + \sum_{k=1}^{M} \lambda_k v_k v_k^*$$
$$= J_{\min} + \sum_{k=1}^{M} \lambda_k |v_k|^2$$

  - $v_k$: the kth component of the vector $\mathbf{v}$

- **Summary**
  - No cross-product term in MSE $\boldsymbol{J}$
  - The error-performance surface is constructed by the transformed coefficient vector $\mathbf{v}$.

## *Steepest Descent*

- **Steepest descent** (SD) algorithm that is an old optimization technique of the **gradient**-based adaptation
  - **Recursive** operation for starting from some **initial value** of the tap-weight vector
  - The **final value** of the **tap-weight** vector that **converges** to the **Wiener solution**
  - **A deterministic feedback** system that fin the minimum point of the ensemble-averaged error-performance surface without knowledge of the surface



## *Steepest-Decent Algorithm*

- Basic Ideal
  - $J(w)$ is a continuously differentiable function.
  - Unconstrained optimization
    - Find an optimal solution $w_o$ that satisfies the following condition

      $J(w_o) \leq J(w)$ for all $w$
  - **Local iterative descent**:

    1. Starting with an initial guess denoted by $\mathbf{w}(0)$, generating a sequence of weight vectors $\mathbf{w}(1), \mathbf{w}(2), \ldots,$ such that the cost function $J(w)$ is reduced at each adaptation cycle of the algorithm

    $$J\big(w(n+1)\big) \leq J\big(w(n)\big)$$

    2. Computing the gradient vector of the cost function $J(\mathbf{w})$ and then using the opposite direction of the gradient vector for further adjustment of $J(\mathbf{w})$

# Steepest-Decent Algorithm (cont'd)

- **Local iterative descent**: (cont'd)
  3. The gradient vector is defined as $\mathbf{g} = \nabla J(\mathbf{w}) = \frac{\partial}{\partial \mathbf{w}} \nabla J(\mathbf{w})$.
  4. Therefore, the steepest-descent algorithm is described as

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \frac{1}{2}\mu \mathbf{g}(n)$$

  – $n$: the adaptation cycle

  – $\mu$: the step-size (The factor ½ can be merged into $\mu$.)

  • In order to evaluate the description of 1., a first-order Taylor series expansion can be applied to find the approximation.

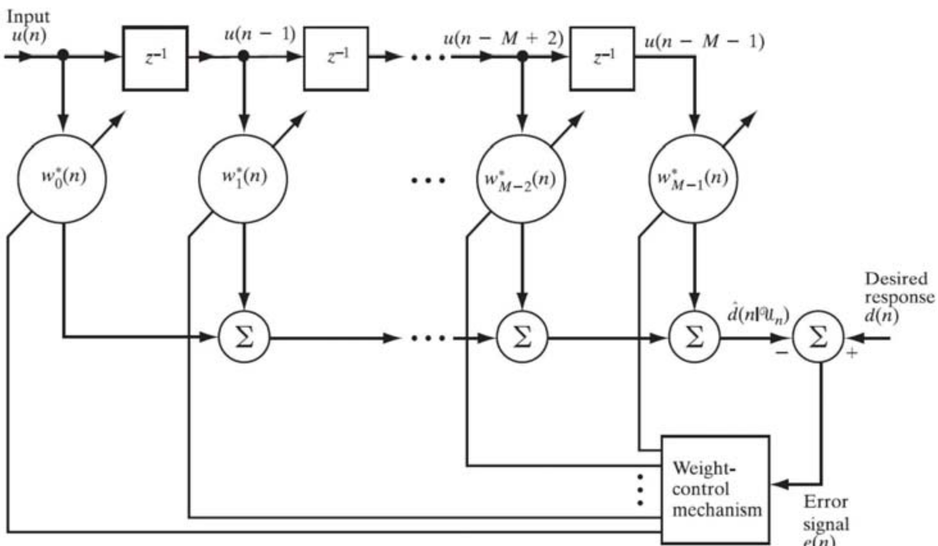
$$J(\mathbf{w}(n+1)) \approx J(\mathbf{w}(n)) + \mathbf{g}^{\mathrm{H}}(n)\delta\mathbf{w}(n)$$

  • Finally, the cost function $J(n)$ progressively decreases, approaching $J_{min}$the as $n \to \infty$.

$$J(\mathbf{w}(n+1)) \approx J(\mathbf{w}(n)) - \frac{1}{2}\mu\|\mathbf{g}(n)\|^2$$



# Steepest Descent Algorithm for Adaptive Transversal Filter




❑ Estimation error $e(n) = d(n) - \hat{d}(n|\mathcal{U}_n)$

$$= d(n) - \mathbf{w}^{\mathrm{H}}(n)\mathbf{u}(n)$$

- Tap-weight vector: $\mathbf{w}(n) = [w_0(n), w_1(n), \ldots, w_{M-1}(n)]^{\mathrm{T}}$
- Tap-input vector: $\mathbf{u}(n) = [u(n), u(n-1), \ldots, (n-M+1)]^{\mathrm{T}}$

# *Steepest Descent Algorithm for Adaptive Transversal Filter (cont'd)*

- MSE at time ***n*** is a quadratic function of the tap-weight vector. Therefore, $J(n)$ can be reformulated as

$$J(n) = \sigma_d^2 - \mathbf{w}^{\mathrm{H}}(n)\mathbf{p} - \mathbf{p}^{\mathrm{H}}\mathbf{w}(n) + \mathbf{w}^{\mathrm{H}}(n)\mathbf{R}\mathbf{w}(n)$$

- The gradient vector of $\nabla J(n)$

$$\nabla J(n) = \begin{bmatrix} \dfrac{\partial J(n)}{\partial a_0(n)} + \mathrm{j}\dfrac{\partial J(n)}{\partial b_0(n)} \\ \dfrac{\partial J(n)}{\partial a_1(n)} + \mathrm{j}\dfrac{\partial J(n)}{\partial b_1(n)} \\ \vdots \\ \dfrac{\partial J(n)}{\partial a_{M-1}(n)} + \mathrm{j}\dfrac{\partial J(n)}{\partial b_{M-1}(n)} \end{bmatrix}$$

$$= -2\mathbf{p} + 2\mathbf{R}\mathbf{w}(n),$$



# *Derivation of* $\nabla J(n)$

- The gradient vector $\nabla J$ is identically zero. That is

$$\nabla_k J = 0, \qquad k = 0, 1, \ldots, M-1$$

- The kth tap-weight is denoted as $w_k = a_k + \mathrm{j}b_k$.
- The $\nabla J_k$ can derived as

$$J = \mathbb{E}[e(n)e^*(n)]$$
$$= \mathbb{E}[|d(n)|^2] - \sum_{k=0}^{M-1} w_k^*\mathbb{E}[u(n-k)d^*(n)] - \sum_{k=0}^{M-1} w_k\mathbb{E}[u^*(n-k)d(n)]$$
$$+ \sum_{k=0}^{M-1}\sum_{i=0}^{M-1} w_k^* w_i\mathbb{E}[u(n-k)u^*(n-i)].$$

$$\nabla_k J = \frac{\partial J}{\partial a_k} + \mathrm{j}\frac{\partial J}{\partial b_k}$$
$$= -2p(-k) + 2\sum_{i=0}^{M-1} w_i r(i-k)$$

$$p(-k) = \mathbb{E}[u(n-k)d^*(n)]$$
$$r(i-k) = \mathbb{E}[u(n-k)u^*(n-i)]$$

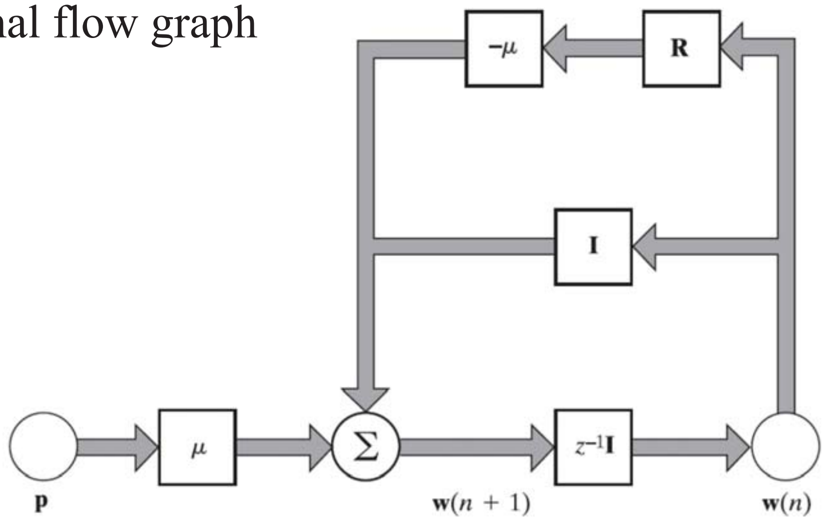
$$\sum_{i=0}^{M-1} w_{oi} r(i-k) = p(-k), \qquad k = 0, 1, \ldots, M-1$$

# *Steepest Descent Algorithm for Adaptive Transversal Filter (cont'd)*

- The updated value of the tap-weight vector can be realized by using the simple recursive operation.

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu[\mathbf{p} - \mathbf{R}\mathbf{w}(n)], \qquad n = 0, 1, 2, \ldots$$

- Signal flow graph






# *Summary*

- The SD algorithm provides a ***simple*** procedure for computing the tap-weight of a Wiener filter, given knowledge of two ensemble-averaged quantities.
  - The correlation matrix **R** of the tap-input vector
  - The cross-correlation vector **P** between the tap-input vector and the desired response
- A critical feature of the SD algorithm
  - Feedback operation → Stability issue
    - The step-size μ
    - The correlation matrix **R**

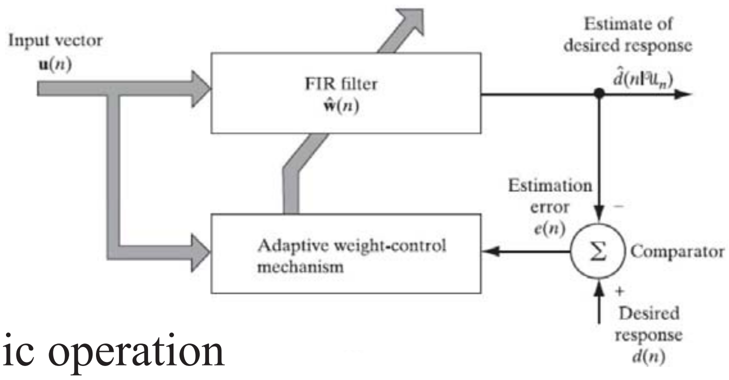
# *Least-Mean-Square Algorithm*

- In the previous, all derivation are based on the following two assumptions.
  - Joint stationarity of the environment for $\mathbf{u}(n)$ and $\mathbf{d}(n)$
  - Information about environment for **R** and **P**
- An important member (LMS) of the family of *stochastic gradient algorithm*
- Different from Wiener filter and Steepest descent
  - **R** and **P**
  - **Matrix inversion $\mathbf{R}^{-1}$**
- LMS algorithm
  - Simple without the requirements of **R** and **P, as well as matrix inversion**



# *LMS Adaptation Algorithm*

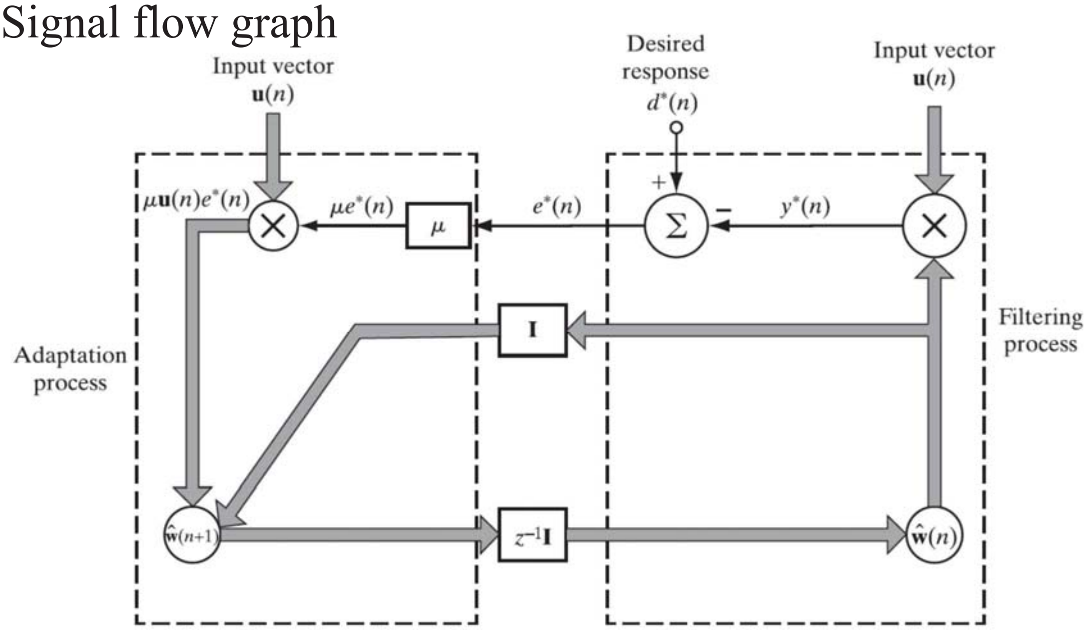
- Block diagram of adaptive transversal filter




- Three basic operation
  - Filtering: $y(n) = \hat{\mathbf{w}}^{\mathrm{H}}(n)\mathbf{u}(n)$
  - Estimation: $e(n) = d(n) - y(n)$
  - Tap-weight adaption: $\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu\mathbf{u}(n)e^*(n)$
- Simplest choice of estimators for **R** and **P**
  - Use instantaneous estimations based on $\mathbf{u}(n)$ and $\mathbf{d}(n)$

# LMS Adaptation Algorithm (cont'd)

- Signal flow graph






# LMS Adaptation Algorithm (cont'd)

- Derivation of the weight updating
  - $\hat{\mathbf{w}} = [\hat{w}_0, \hat{w}_1, \dots, \hat{w}_{M-1}]^T$
  - $\mathbf{u} = [u_n, u_{n-1}, \dots, u_{n-M+1}]^T$
  - A **gradient noise** buried in each tap weight
  - $J(n) \rightarrow J(\infty)$ as $n \rightarrow \infty$; $\therefore J(\infty) \rightarrow J_{min}$

## LMS Adaptation Algorithm (cont'd)

- LMS adaptation
  - An initial guess $\hat{\mathbf{w}}(0)$
  - **Feedback loop** acting around the tap weights like a **low-pass filter**
  - The **average** time constant $\propto 1/\mu$
  - Complexity per iteration
    - $(2M+1)$ complex MULs
    - $2M$ complex ADDs
- Excess of minimum MSE $J_{min}$
  - $J_{ex}(n) = J(n) - J_{min}$
- Misadjustment $\mathcal{M}$: the ratio of $J_{ex}(\infty)$ to $J_{min}$
  - $\mathcal{M} = J_{ex}(\infty)/J_{min}$



## LMS Adaptation Algorithm (cont'd)

- Simple working rules
  - $0 < \mu < \frac{2}{\lambda_{max}} < \frac{2}{\mathrm{tr}[\mathbf{R}]}$
    - $\mathbf{R}$: autocorrelation of tap-input vector $\mathbf{u}$
    - $\lambda_{max}$: the maximum eigenvalue of eigen-decomposition for $\mathbf{R}$
    - $$\begin{aligned}\mathrm{tr}[\mathbf{R}] &= Mr(0)\\ &= \sum_{k=0}^{M-1} \mathbb{E}[|u(n-k)|^2]\\ &= \mathbb{E}[\|\mathbf{u}(n)\|^2].\end{aligned}$$
  - $0 < \mu < \frac{2}{\textbf{tap input power}}$
  - $\mathcal{M} \approx \frac{\mu}{2}\sum_{i=1}^{M}\lambda_i = \frac{\mu}{2}(\textbf{tap input power})$

# LMS Adaptation Algorithm (cont'd)

- Define an average eigenvalue of **R**

  $\lambda_{avg} = \frac{1}{M}\sum_{i=1}^{M}\lambda_i$

- Define average time-constant

  $\tau_{MSE,avg} \approx \frac{1}{2\mu\lambda_{avg}}$

- Approximation of $\mathcal{M}$ as the following equation

  $\mathcal{M} \approx \frac{\mu M \lambda_{avg}}{2} = \frac{M}{4\tau_{MSE,avg}}$; $\left[\because \mathcal{M} \approx \frac{\mu}{2}\sum_{i=1}^{M}\lambda_i = \frac{\mu}{2}(\textbf{tap input power})\right]$

- Performance index
  - $\mathcal{M}$ vs. $M$
  - Settling time vs. $\tau_{MSE,avg}$
  - $\mathcal{M}$ vs. the settling time ($\tau_{MSE,avg}$)
  - $\mathcal{M}$ vs. $\mu$



# Summary of LMS Algorithm

*Parameters*: $M$ = number of taps (i.e., filter length)
$\mu$ = step-size parameter

$$0 < \mu < \frac{2}{\lambda_{\max}},$$

where $\lambda_{\max}$ is the maximum value of the correlation matrix of the tap inputs $u(n)$ and the filter length $M$ is moderate to large.

*Initialization*: If prior knowledge of the tap-weight vector $\hat{\mathbf{w}}(n)$ is available, use it to select an appropriate value for $\hat{\mathbf{w}}(0)$. Otherwise, set $\hat{\mathbf{w}}(0) = \mathbf{0}$.

*Data*:

- Given $\mathbf{u}(n)$ = $M$-by-1 tap-input vector at time $n$
  $= [u((n), u(n\text{-}1)), \ldots, u(n-M+1)]^T$
  $d(n)$ = desired response at time $n$.
- To be computed:
  $\hat{\mathbf{w}}(n+1)$ = estimate of tap-weight vector at time $n+1$.

*Computation*: For $n = 0, 1, 2, \ldots$, compute

$$e(n) = d(n) - \hat{\mathbf{w}}^H(n)\mathbf{u}(n)$$
$$\hat{\mathbf{w}}(n+1) = \hat{\mathbf{w}}(n) + \mu\mathbf{u}(n)e^*(n).$$

# Application of LMS Algorithm

- Application-**1**: Canonical model of the complex LMS algorithm
  - Tap input vector: $\mathbf{u}(n) = \mathbf{u}_I(n) + j\mathbf{u}_Q(n)$
  - Desired response: $d(n) = d_I(n) + jd_Q(n)$
  - Tap weight vector: $\hat{\mathbf{w}}(n) = \hat{\mathbf{w}}_I(n) + j\hat{\mathbf{w}}_Q(n)$
  - Transversal filter output: $y(n) = y_I(n) + jy_Q(n)$

$$y_I(n) = \hat{\mathbf{w}}_I^{\mathrm{T}}(n)\mathbf{u}_I(n) - \hat{\mathbf{w}}_Q^{\mathrm{T}}(n)\mathbf{u}_Q(n)$$

$$y_Q(n) = \hat{\mathbf{w}}_I^{\mathrm{T}}(n)\mathbf{u}_Q(n) + \hat{\mathbf{w}}_Q^{\mathrm{T}}(n)\mathbf{u}_I(n)$$

  - Estimation error: $e(n) = e_I + je_Q(n)$

$$e_I(n) = d_I(n) - y_I(n)$$
$$e_Q(n) = d_Q(n) - y_Q(n)$$

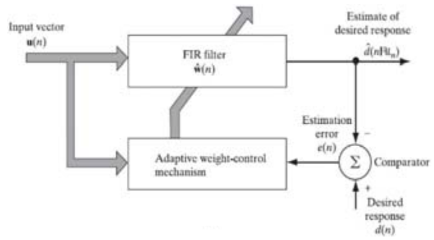
  - Updating of tap weights:

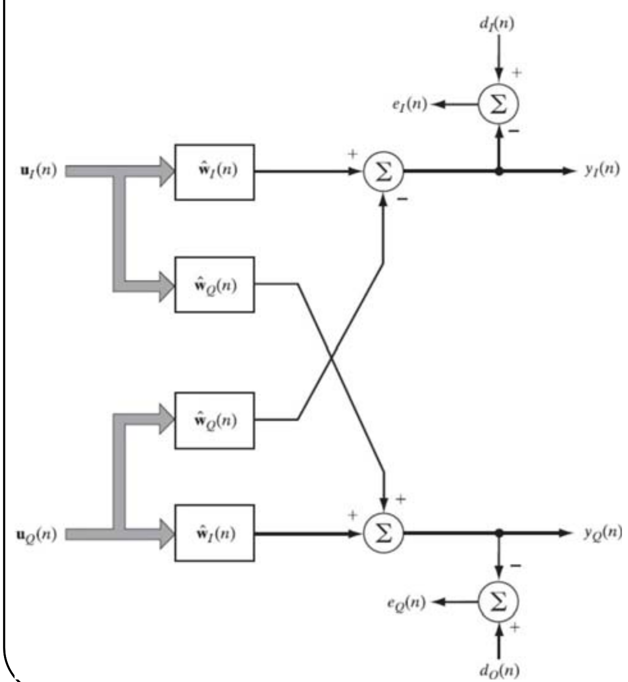
$$\hat{\mathbf{w}}_I(n+1) = \hat{\mathbf{w}}_I(n) + \mu[e_I(n)\mathbf{u}_I(n) - e_Q(n)\mathbf{u}_Q(n)]$$
$$\hat{\mathbf{w}}_Q(n+1) = \hat{\mathbf{w}}_Q(n) + \mu[e_I(n)\mathbf{u}_Q(n) + e_Q(n)\mathbf{u}_I(n)]$$

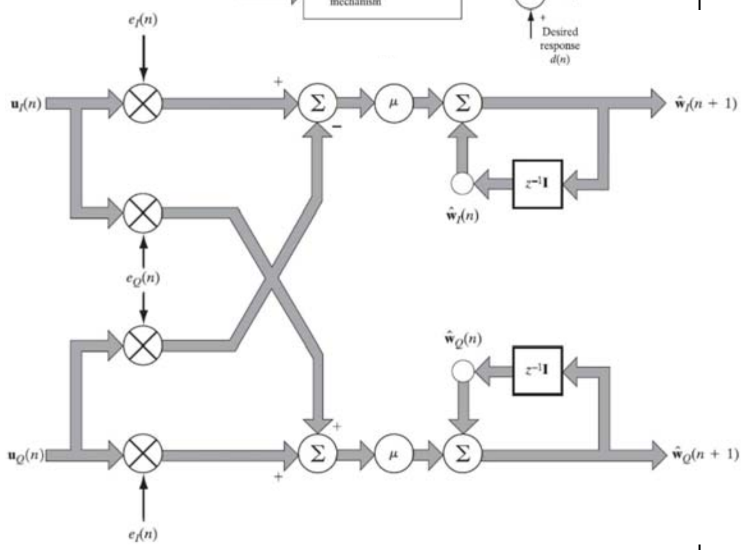


# Application of LMS Algorithm (cont'd)

- Signal flow graph





# Application of LMS Algorithm (cont'd)

- Application field
  - Adaptive equalization of a communication system for QPSK and QAM modulation over a dispersive channel
- Two limited factors for data transmission through the **dispersive** channel (BW-limited channel)
  - Inter-symbol interference (ISI)
  - Thermal noise
- In general, both adaptive filters are applied to combat the dispersive channel for cancelling the ISI.
  - Feed-forward equalizer (FFE)
  - Feed-backward equalizer (FBE)



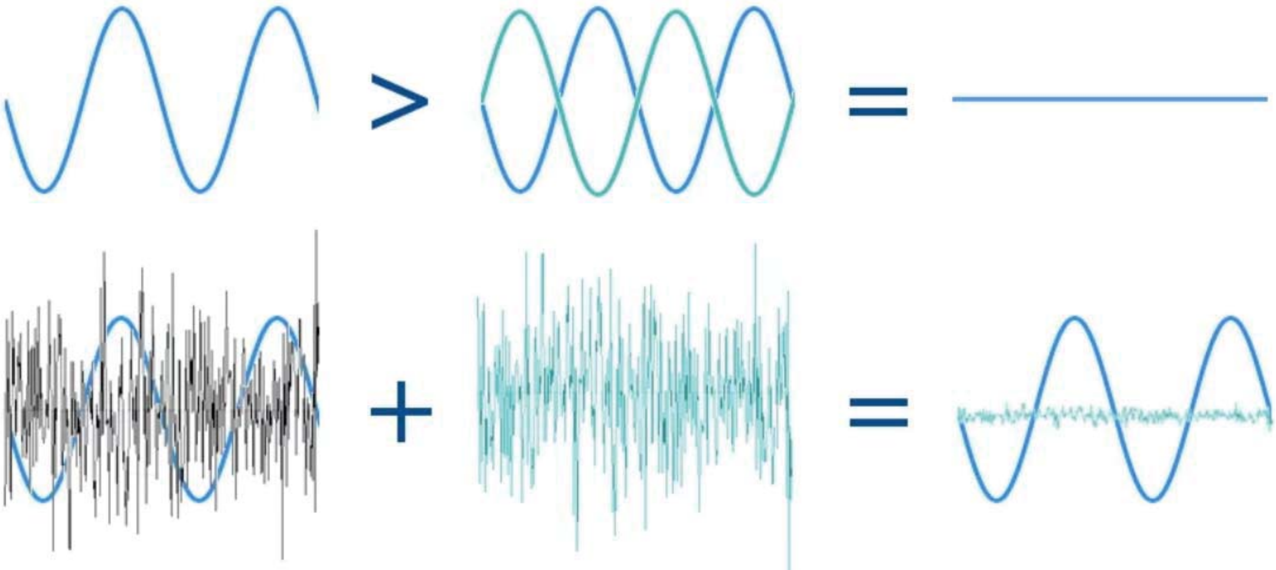
# Application of LMS Algorithm (cont'd)

- Application-**2**: Ear phones with ANC

- Why We Need ANC for Earphones?
  - Do NOT harm to our listening
  - Provide high-quantity listening for voice, speech, music, etc.

## *How Does ANC Work?*

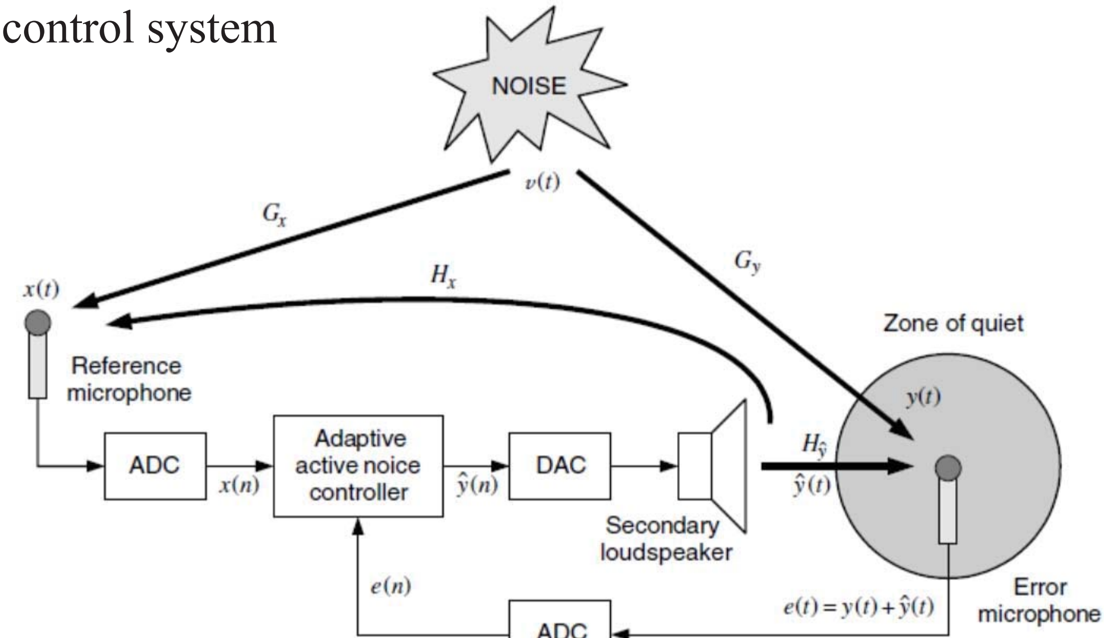
- ❑ Addition of two waves which are equal amplitude and opposite phase





## *Active Noise Cancellation*

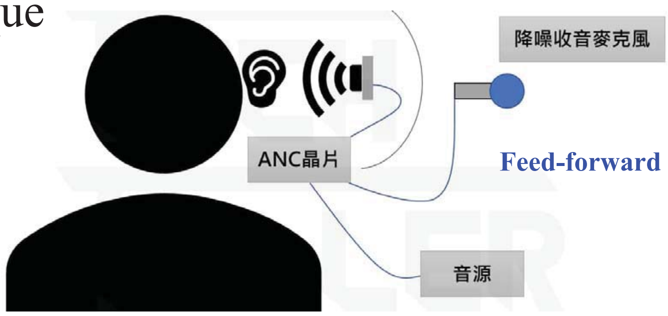
- ❑ Block diagram of the basic components of an active noise control system

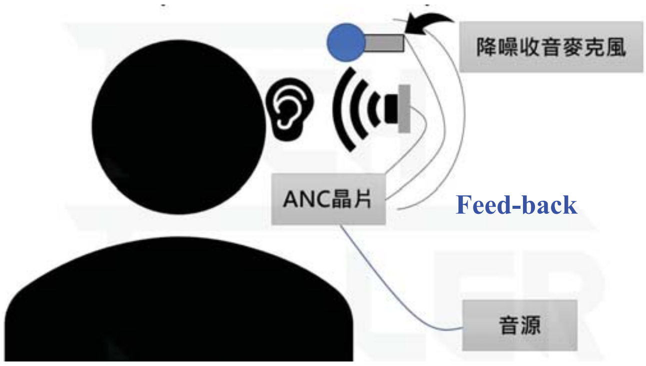


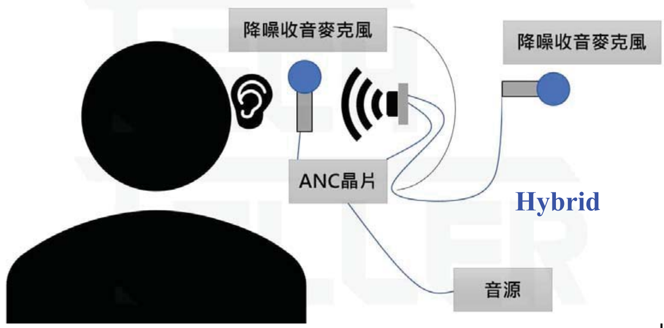

Ref.: D. G. Manolakis, et. al, *Statistical and Adaptive Signal Processing*, Atech House, INC., 2005.

# Adaptive Noise Cancellation (ANC)

- ❑ Three types of ANC technique
  - ■ Feed-forward ANC
  - ■ Feed-back ANC
  - ■ Hybrid ANC



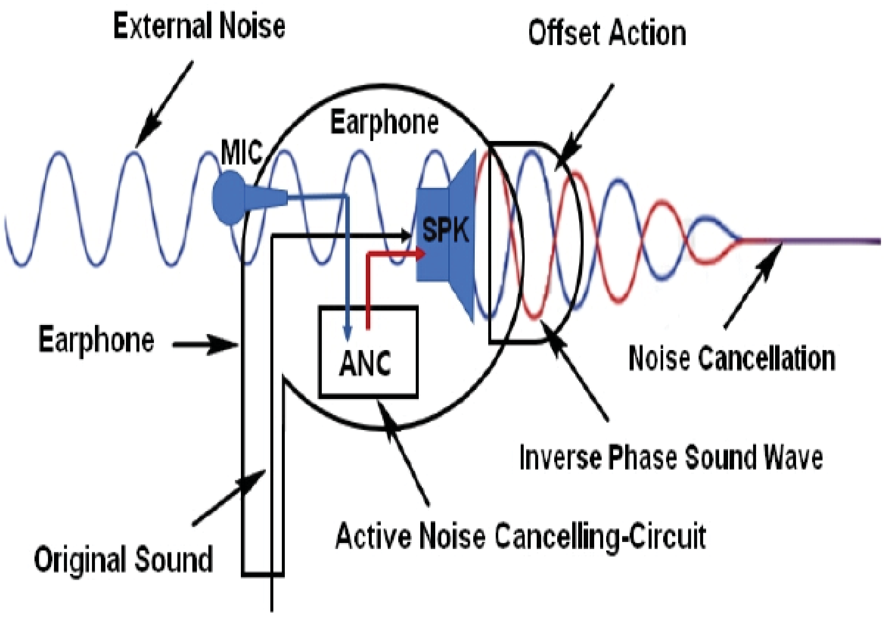




Ref.: https://techteller.com/sci/active-noise-cancellation-headphone/



# ANC (cont'd)

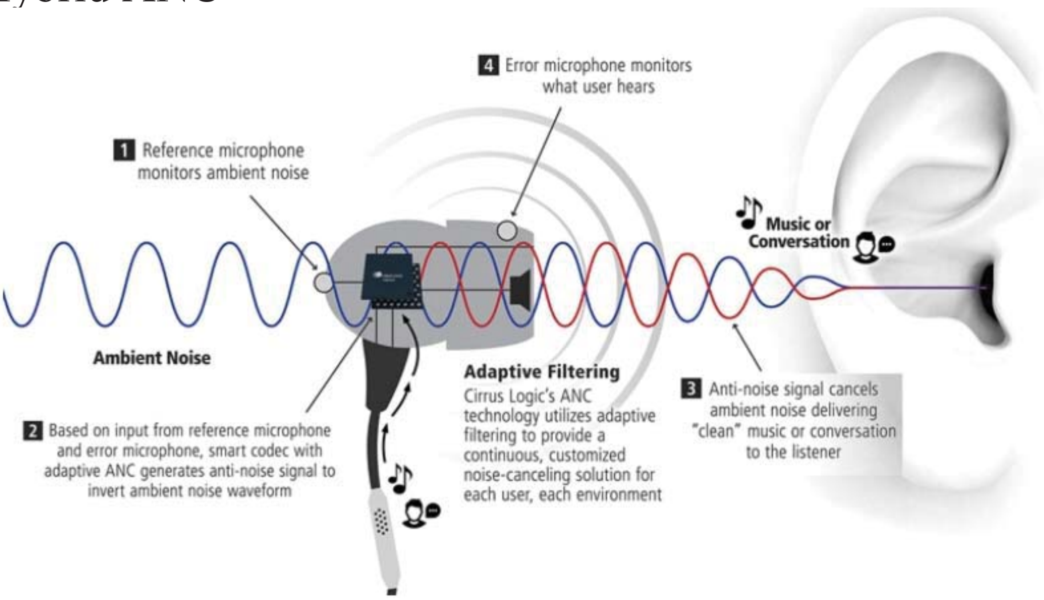
- ❑ Feed-forward ANC




Ref.: IJETT, '20

# ANC (cont'd)

❑ Hybrid ANC

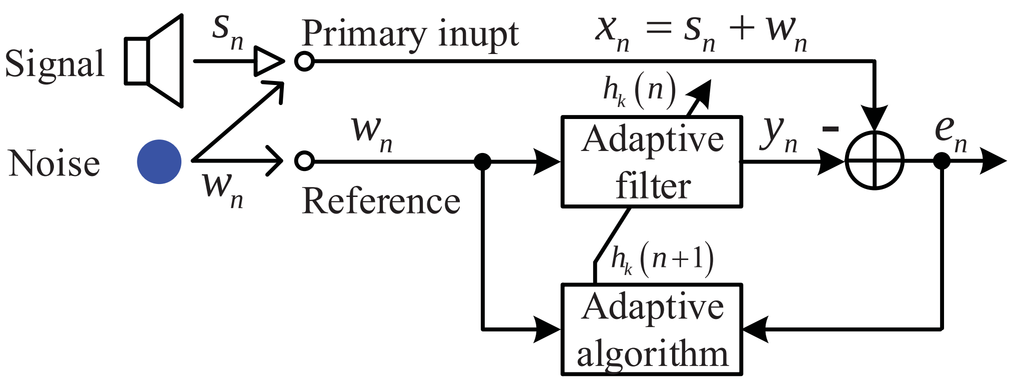



**Ref.: https://www.analogictips.com/active-noise-cancellation-part-2-implementation-faq/**



# Signal Model of ANC System

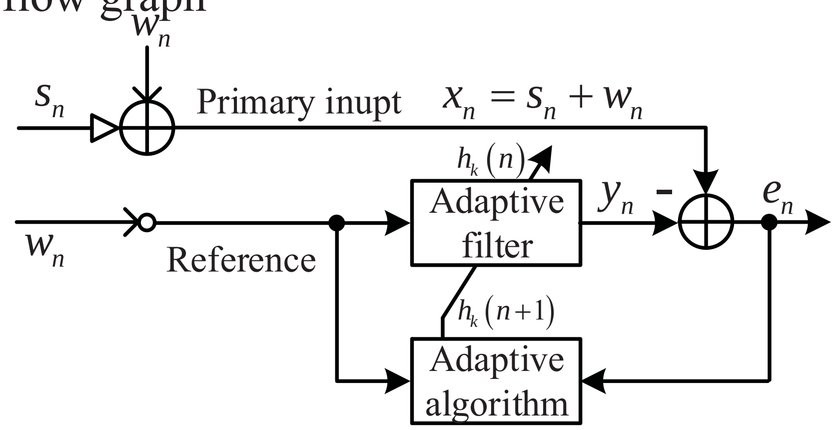
❑ Signal model




- $y_n = \widehat{w}_n \approx w_n$
- $e_n = x_n - y_n = s_n + w_n - \widehat{w}_n = \hat{s}_n \approx s_n$

# Signal Flow Graph of ANC Signal Model

❑ Signal flow graph

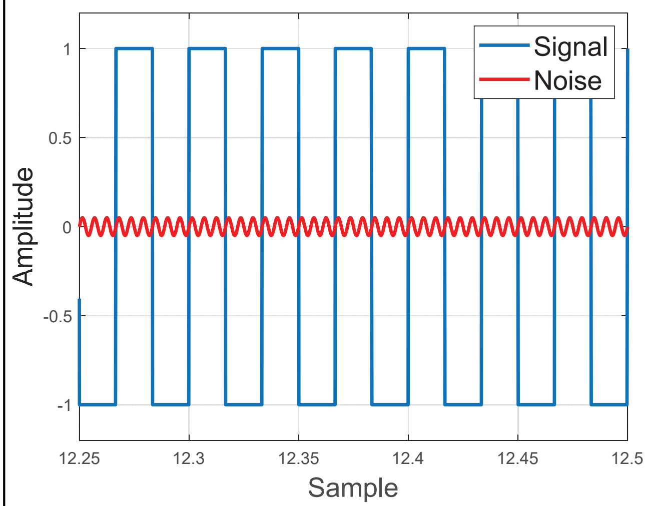


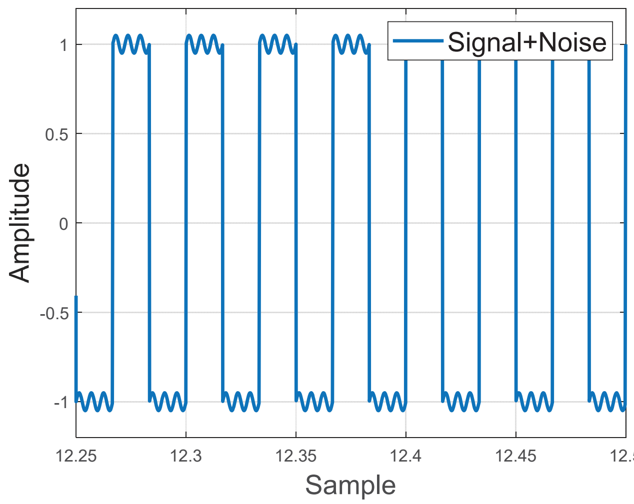

- Signal: square wave ($f_{signal}$ = 30 Hz)
- Noise: sine wave ($f_{noise}$ = 180 Hz)
- Sampling period: $T_s$ = 0.1 ms ($f_s$ = 10 KHz)



# Simulation Results

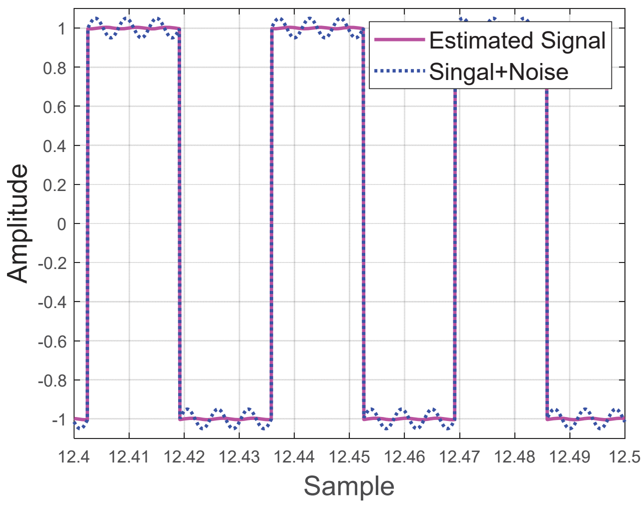
❑ Signal and noise models

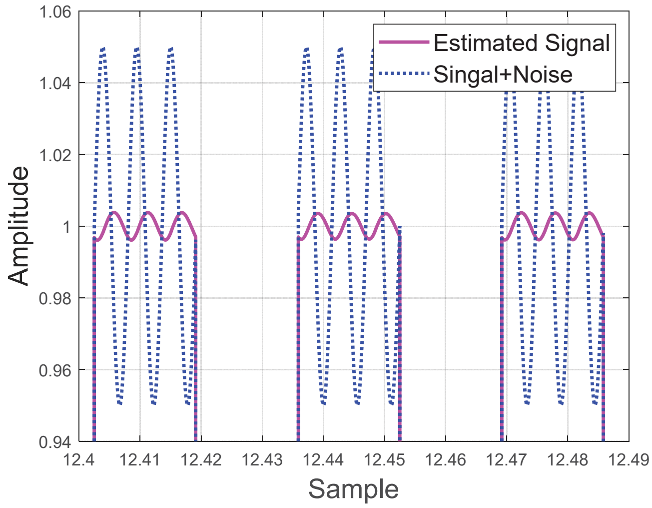


# Simulation Results (cont'd)

- ANC outputs

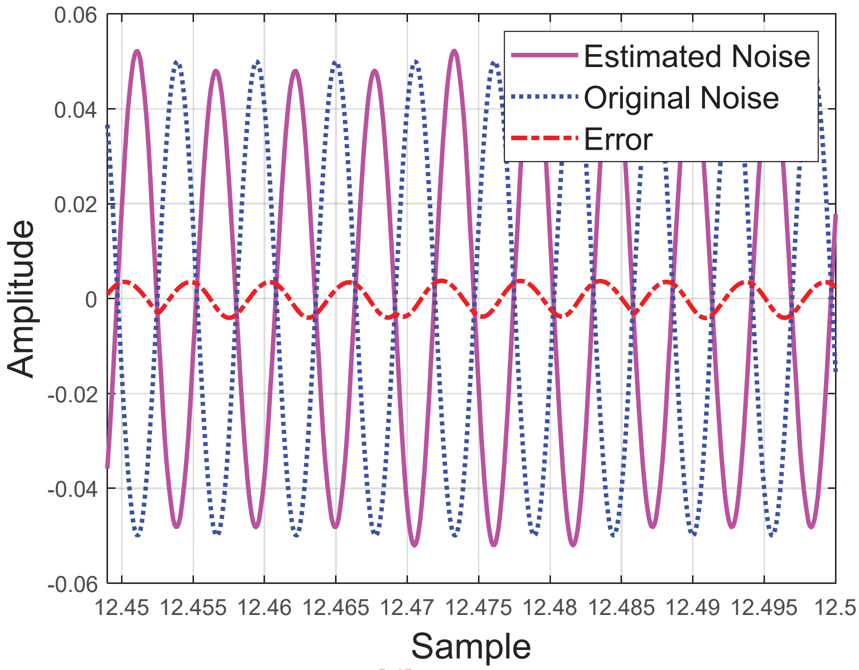





# Simulation Results (cont'd)

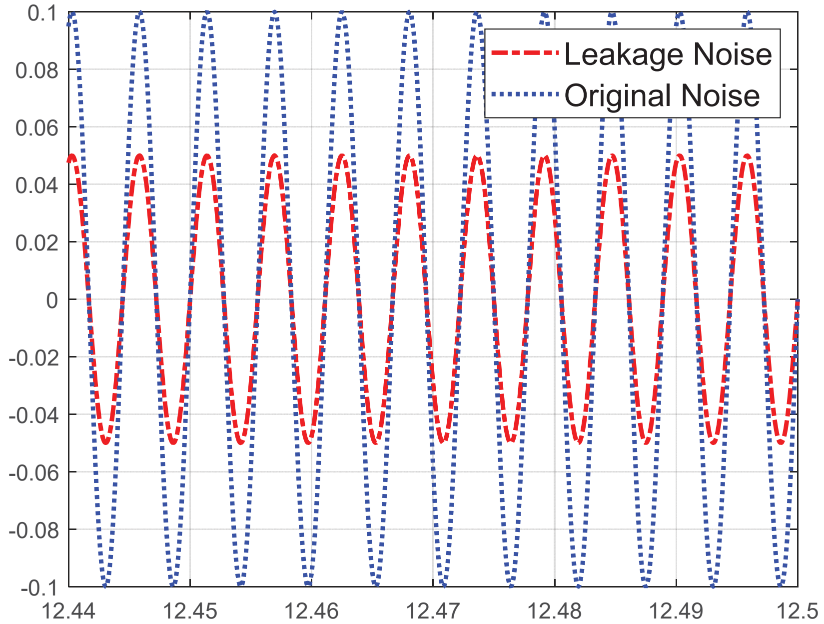
- Error between the original and the estimated noises




$Error \leq \pm 0.004$

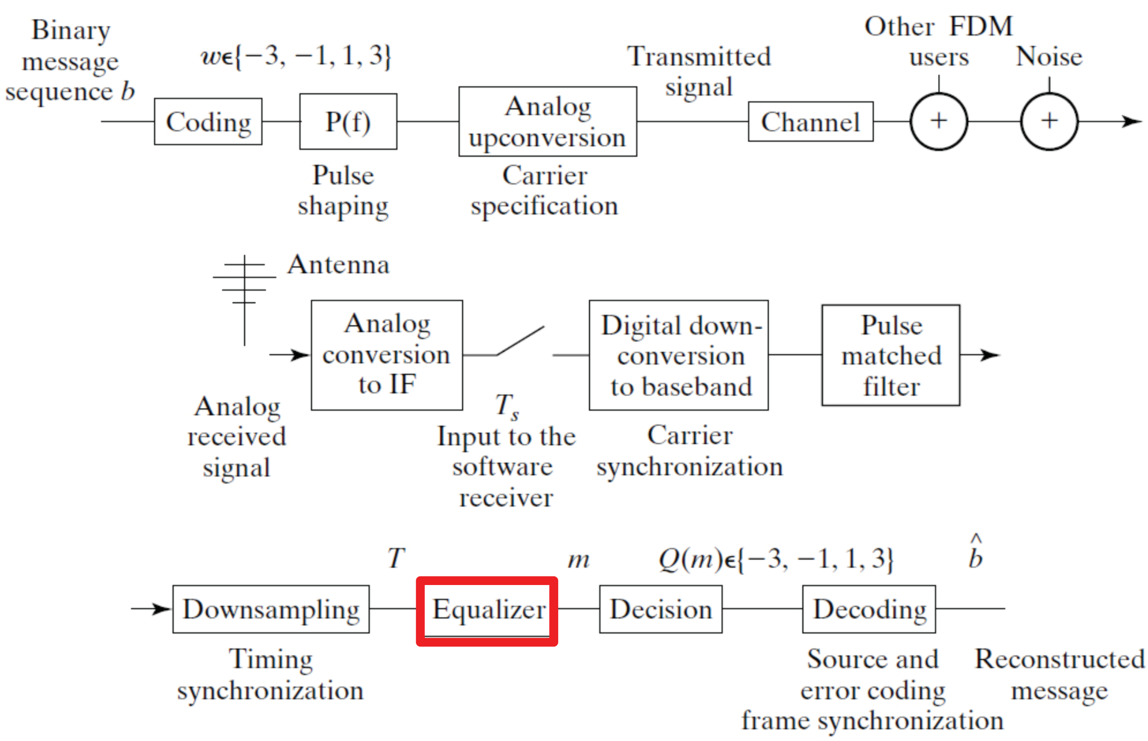
## Simulation Results (cont'd)

- Consideration of the leakage noise from earphones

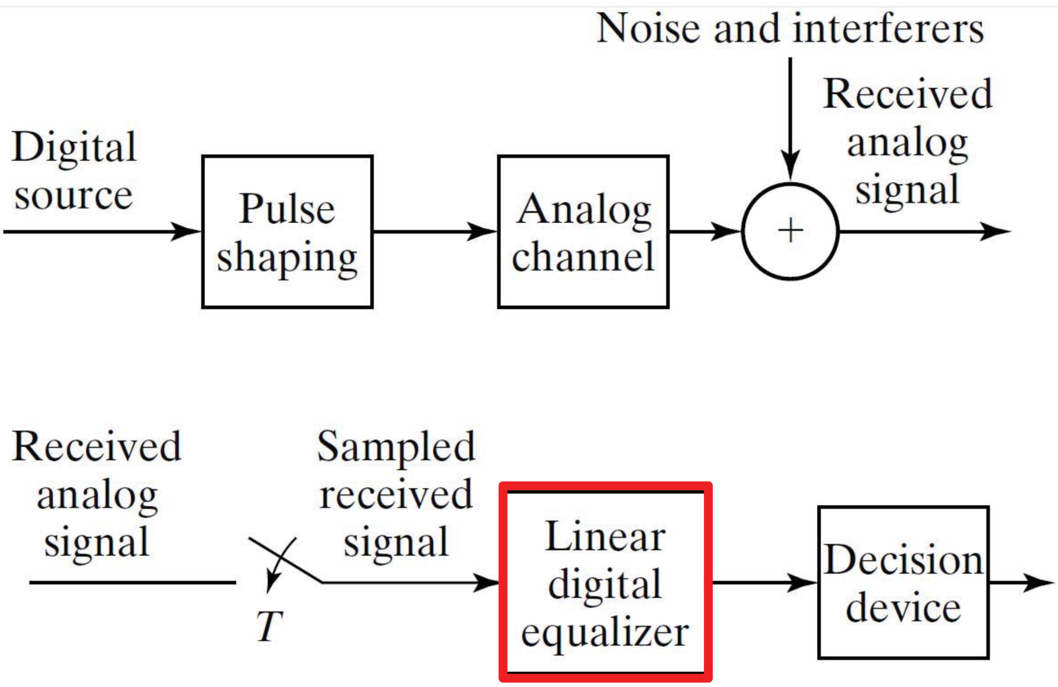





## Communication System



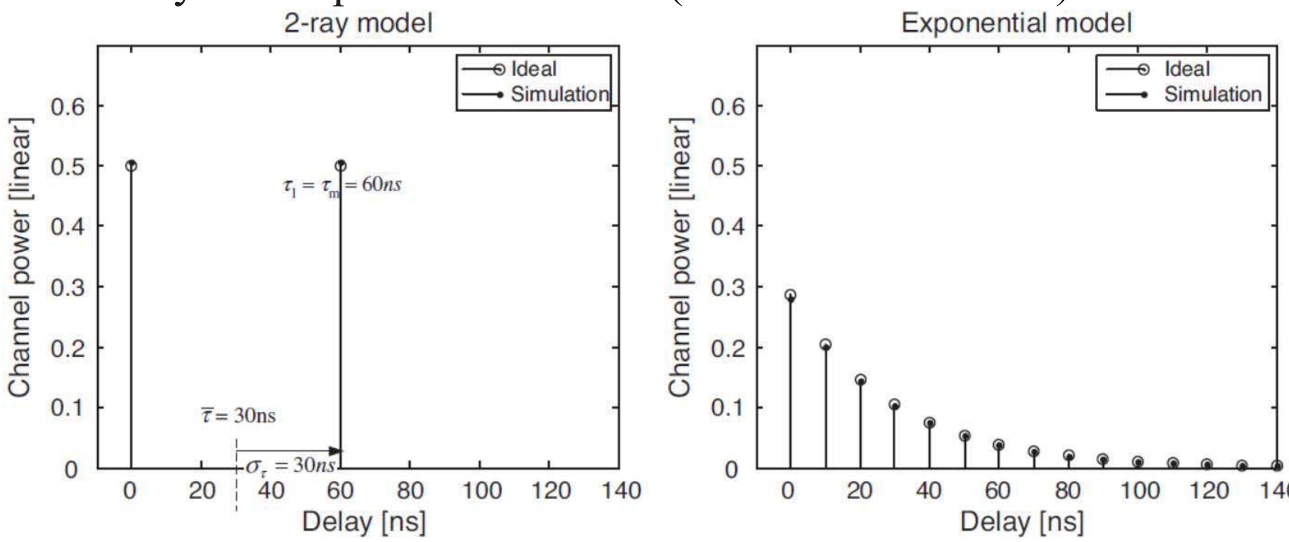
# *Baseband Linear Equalizer*






# *General Indoor Channel Models*

❑ 2-ray and exponential models (indoor environment)




- The channel power delay profile: $P(\tau) = \frac{1}{\tau_d} e^{-\tau/\tau_d}$
  - $\tau_m$ (the max. excess delay), $\tau_d$ (the RMS delay spread), $\bar{\tau}$ (the mean excess delay)

## General Indoor Channel Models (cont'd)

- $\bar{\tau} = \sigma_\tau = \tau_d$ in the exponential model
- $\tau_m = -\tau_d \ln A$, where A is a ratio of non-negligible path power to the first path power.

$$A = P(\tau_m)/P(0) = exp(-\tau_m/\tau_d)$$

- Therefore, the discrete-time model with a sampling period of $T_s$ is given as

$$P(p) = \frac{1}{\sigma_\tau} e^{-pT_s/\sigma_\tau}, \qquad p = 0, 1, 2, \dots, p_{max}$$

  - $p$: the discrete-time index
  - $p_{max}$: the last path, i.e., $p_{max} = [\tau_m/T_s]$
- A total power of the PDP is derived as

$$P_{total} = \sum_{p=0}^{p_{max}} P(p) = \frac{1}{\sigma_\tau} \cdot \frac{1 - e^{-(p_{max}+1)T_s/\sigma_\tau}}{1 - e^{-T_s/\sigma_\tau}}$$



## General Indoor Channel Models (cont'd)

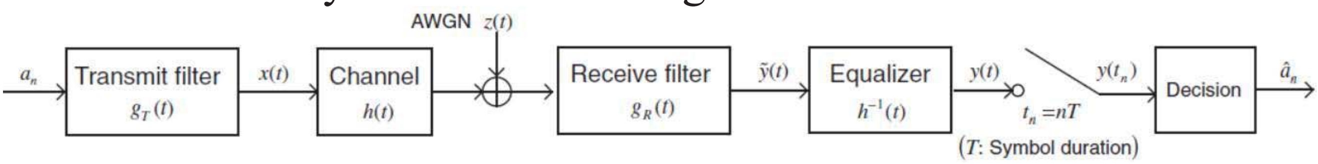
- The normalize PDP is given as

$$P(p) = P(0)e^{-pT_s/\sigma_\tau}, \quad p = 0, 1, 2, \dots, p_{max}$$

  - $P(0)$: the first path power and $P(0) = 1/(P_{total} \cdot \sigma_\tau)$

# Single-Carrier Baseband Transmission: System Model

- Baseband system model of single carrier communication




- The output of the equalizer can be expressed as

$$y(t) = \sum_{m=-\infty}^{\infty} a_m g(t - mt) + z(t)$$

  - $g(t)$: the impulse response of overall end-to-end system
  - $g(t) = g_T(t) * h(t) * \ g_R(t) * h^{-1}(t)$
- Equalizer
  - Design to compensate the effect of channel



# Single-Carrier Baseband Transmission: System Model

- The sampled output signal of the equalizer can be expressed as (ignore the noise term)

$$y(t_n) = \sum_{m=-\infty}^{\infty} a_m g\big((n-m)T\big), \qquad t_n = nT$$

- Considering the nth sample to detect $a_n$, the $y(t_n)$ can be written as

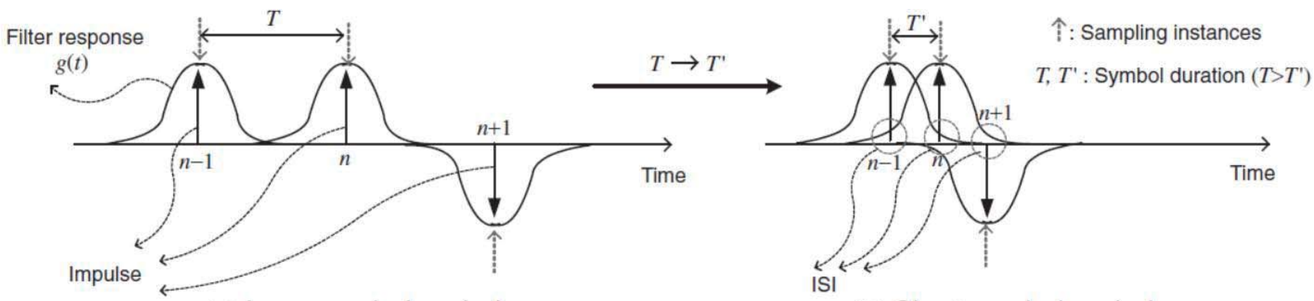
$$y(t_n) = a_n g(0) + \sum_{m=-\infty, m\neq n}^{\infty} a_m g\big((n-m)T\big)$$

  - $g(0)$ is NOT time-limited due to the finite channel BW.
  - $g\big((n-m)T\big) \neq 0$ for all $m \neq n$
  - The 2nd term is called as **inter-symbol interference (ISI)** that destroys $a_n$.

# Single-Carrier Baseband Transmission: System Model (cont'd)

- The ISI is caused by a trail of the overall impulse response that could degrade the system performance.
- In a practical system, the transmit filter and receiver filter must be design to minimize or completely eliminate the ISI.

❑ Relationship between ISI and symbol/sample period




(a) Long symbol period (b) Short symbol period



# Single-Carrier Baseband Transmission: System Model (cont'd)

❑ ISI and Nyquist criterion

- ISI can be completely eliminated by fulfilling the following condition on the overall impulse response.

$$g(nT) = \delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

- Its frequency response is given as

$$\sum_{i=-\infty}^{\infty} G\left(f - \frac{i}{T}\right) = T$$

- Nyquist criterion
  - An ISI-free communication even with a short symbol period T for high rate transmission in a single-carrier system

# Single-Carrier Baseband Transmission: System Model (cont'd)

- Nyquist filter
  - Satisfy the Nyquist criterion
  - An ideal LPF, which has a **sinc** function-type of impulse response, rectangular pulse type of frequency response as

$$G(f) = \frac{1}{2W} rect\left(\frac{f}{2W}\right) = \begin{cases} T, & |f| \leq \frac{1}{2T} \\ 0, & |f| > \frac{1}{2T} \end{cases}$$

  - Nyquist BW: $W = R/2 = 1/2T$
  - $R$: Nyquist rate
  - Nyquist BW $W$ is the minimum possible BW that is required to realize the data rate $R$ without ISI.



# Single-Carrier Baseband Transmission: System Model (cont'd)

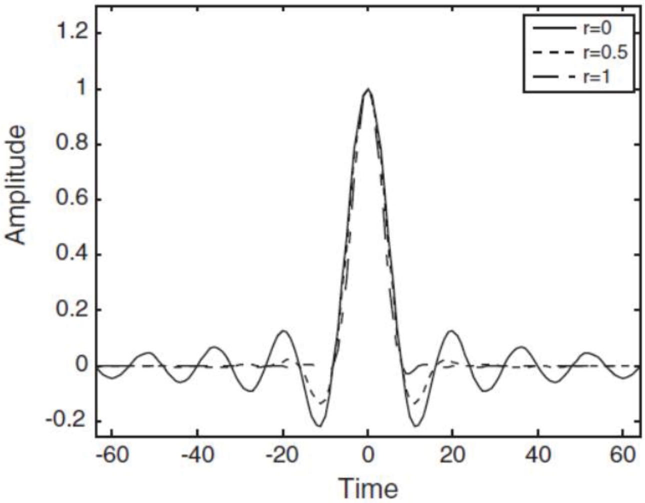
- Raised-cosine filter
  - Physically realizable Nyquist filter

$$G_{RC}(f) = \begin{cases} T, & |f| \leq \frac{1-r}{2T} \\ \frac{T}{2}\left\{1 + cos\frac{\pi T}{r}\left(|f| - \frac{1-r}{2T}\right)\right\}, & \frac{1-r}{2T} < |f| \leq \frac{1+r}{2T} \\ 0, & |f| > \frac{1+r}{2T} \end{cases}$$

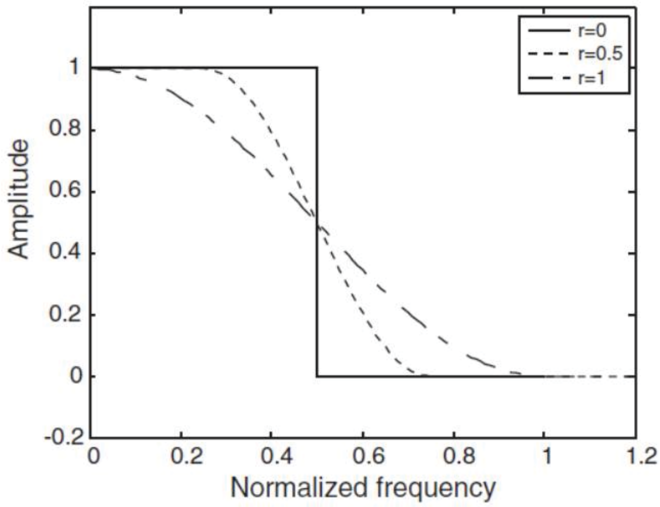
  - $r$: the roll-off factor ($0 \leq r \leq 1$)
  - $r = 0$: The raised-cosine filter is an ideal LPF.
  - $r = 1$: The raised-cosine filter occupies twice the Nyquist BW.

## *Single-Carrier Baseband Transmission: System Model (cont'd)*

❑ Raised-cosine filter




(a1) Impulse responses of raised cosine filters




(a2) Frequency responses of raised cosine filters



## *Single-Carrier Baseband Transmission: System Model (cont'd)*

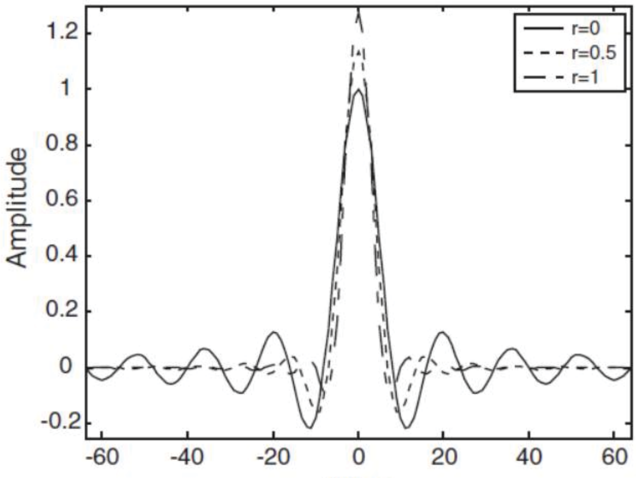
❑ Square-root raised-cosine filter

- In special case, $G_R(f) = G_T^*(f)$
- Then, $G_{RC}(f) = |G_T(f)|^2$ or $G_T(f) = \sqrt{G_{RC}(f)}$
- Finally, the frequency response of Square-root raised-cosine filter is given as

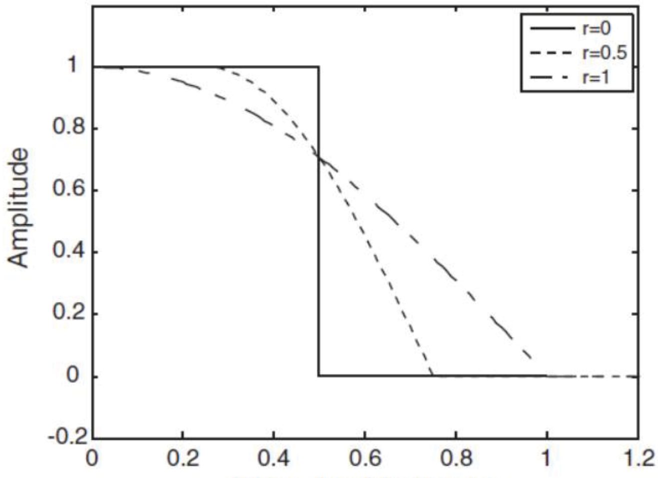
$$G_{SRRC}(f) = \begin{cases} \sqrt{T}, & |f| \le \dfrac{1-r}{2T} \\ \sqrt{\dfrac{T}{2}\left\{1 + cos\dfrac{\pi T}{r}\left(|f| - \dfrac{1-r}{2T}\right)\right\}}, & \dfrac{1-r}{2T} < |f| \le \dfrac{1+r}{2T} \\ 0, & |f| > \dfrac{1+r}{2T} \end{cases}$$

# *Single-Carrier Baseband Transmission: System Model (cont'd)*

- ❑ Square-root raised cosine filter




(b1) Impulse responses of square-root raised cosine filters




(b2) Frequency responses of square-root raised cosine filters



# *Linear Equalizer*

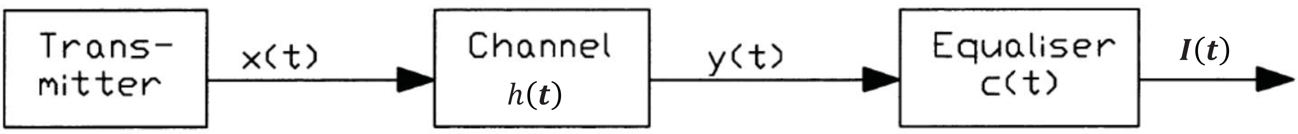
- ❑ Zero-forcing equalizer
  - ■ ISI-free and Nyquist criterion

$$g(nT) = \delta[n] = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

  - ■ $g(nT) = g_T(nT) * h(nT) * \ g_R(nT) * h^{-1}(nT)$
- ❑ The z-transform of an equalizer is equivalent to
  - ■ $C(z) = 1/H(z)$
  - ■ The channel response is finite length described as

$$H(z) = h(0) + h(1)z^{-1} + \cdots + h(n)z^{-n}$$

    - $h(0), h(1), \ldots, h(n)$: the coefficients of the FIR filter $H(z)$



# Linear Equalizer (cont'd)

- The z-transform of the equalizer is derived as

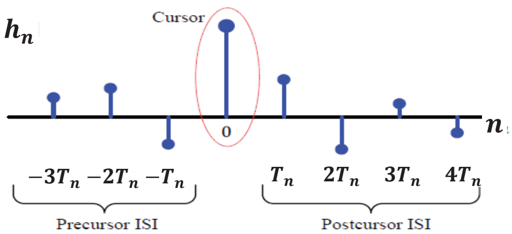
  $C(z) = 1/\{h(0) + h(1)z^{-1} + \cdots + h(n)z^{-n}\}$

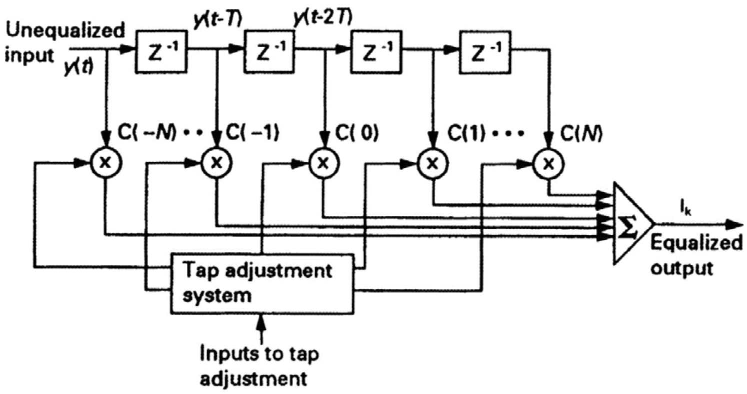
  $C(z) = c(0) + c(1)z^{-1} + c(2)z^{-2} + \cdots$

  - $c(0)$, $c(1)$, $c(n)$, ... : an infinite length sequence of the equalizer coefficients

- General linear equalizer

$$I_k = \sum_{i=-N}^{N} c(i)y(t-iT)$$




Ref.: https://wirelesspi.com/how-decision-feedback-equalizers-dfe-work/



# Least Mean Squared (LMS) Equalizer

- LMS
  - Minimize the joint effects of ISI and noise at the equalizer output
  - Tolerate higher ISI and have faster convergence than ZF
- Mean squared distortion (MSD)

$$MSD = \frac{1}{I^2(0)} \sum_{\substack{n=-\infty \\ n\neq 0}}^{\infty} I^2(n)$$

- Recall the output of equalizer

$$I(n) = y(n) * c(n) = \sum_{i=-N}^{N} c_i y(n-i)$$

  - $I(0) = \sum_{i=-N}^{N} c_i y(i)$

## Least Mean Squared Equalizer (cont'd)

- The minimization of the effects of ISI is minimizing the mean squared error (MSE).

$$\epsilon = \sum_{\substack{n=-\infty \\ n\neq 0}}^{\infty} I^2(n) = \left(\sum_{n=-\infty}^{\infty} I^2(n)\right) - I^2(0)$$

  - A quadratic function of the equalizer coefficients and it has a minimum.
- Then, we have

$$\frac{\partial}{\partial c_j}\epsilon = \left(\sum_{n=-\infty}^{\infty} 2I(n)\frac{\partial I(n)}{\partial c_j}\right) - 2I(0)\frac{\partial I(0)}{\partial c_j} = 0$$

$$\sum_{n=-\infty}^{\infty} I(n)y(n-i) = I(0)y(-i), i = -N, \ldots, N$$

  - Obviously, the main tap $I(0)$ does not play any role in minimizing the MSE of the system response.
  - For simplicity, the main tap is set to $I(0) = 1$.



## Least Mean Squared Equalizer (cont'd)

- Correlation matrix (substituting $I(n)$ into above equation)

$$\sum_{n=-\infty}^{\infty}\sum_{k=-N}^{N} c(k)\, y(n-k)y(n-i) = y(-i)$$

$$\sum_{k=-N}^{N} c(k) \sum_{n=-\infty}^{\infty} y(n-k)y(n-i) = y(-i)$$

  - Define the correlation coefficients of the received signal $y(n)$

$$b(i,k) = \sum_{n=-\infty}^{\infty} y(n-k)y(n-i)$$

  - Then,

$$\sum_{k=-N}^{N} c(k)\, b(i,k) = y(-i), i = -N, \ldots, N$$

## *Least Mean Squared Equalizer (cont'd)*

- (2N+1) linear equations minimizing the MSE term

- The convenient matrix form is expressed as

$$\underline{\mathbf{B}} \cdot \underline{\mathbf{c}} = \underline{\mathbf{y}}$$

  - $\underline{\mathbf{B}}$: [2N+1]×[2N+1] matrix
  - $\underline{\mathbf{c}}$: [2N+1]×1 vector
  - $\underline{y}$: [2N+1]×1 vector

- Assuming a stationary received signal $y(n)$, its correlation coefficient will not be dependent on the actual values of the indexes $(i, k)$, only their differences.



## *Least Mean Squared Equalizer (cont'd)*

- Its detail matrix form is given as

$$\begin{bmatrix} b(0) & b(1) & b(2) & \cdots & b(2N) \\ b(1) & b(0) & b(1) & \cdots & b(2N-1) \\ b(2) & b(1) & b(0) & \cdots & b(2N-2) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b(2N) & b(2N-1) & b(2N-2) & \cdots & b(0) \end{bmatrix} \begin{bmatrix} c(-N) \\ c(-N+1) \\ c(-N+2) \\ \vdots \\ c(N) \end{bmatrix} = \begin{bmatrix} y(N) \\ y(N-1) \\ y(N-2) \\ \vdots \\ y(-N) \end{bmatrix}$$

  - Symmetric and identical within the main diagonals

- The ideal system response has a unity value at n=0, i.e.,

$$I(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

- Therefore, the error sample $e(n)$ is given as

$$e(n) = \begin{cases} I(0) - 1, & n = 0 \\ I(n), & n \neq 0 \end{cases}$$

# Least Mean Squared Equalizer (cont'd)

- Then,

$$\sum_{n=-\infty, n\neq 0}^{\infty} I(n)y(n-i) + I(0)y(-i) = 0$$

and hence,

$$\sum_{\substack{n=-\infty,\\ n\neq 0}}^{\infty} I(n)y(n-i) = \sum_{\substack{n=-\infty\\ n\neq 0}}^{\infty} e(n)y(n-i) = 0, j = -N, \dots, N$$

  - The error sample $e(n)$ is uncorrelated with the equalizer input signal $y(n)$.

❑ The optimal solution of $\underline{\mathbf{c}}$ vector

$$\underline{\mathbf{c}}^{opt} = \underline{\mathbf{B}}^{-1}\underline{\mathbf{y}}$$

- In practical system, the steepest descent or the gradient algorithm is a useful approach using iterative procedure.



# Least Mean Squared Equalizer (cont'd)

- The (2N+1) gradient vector $\underline{\boldsymbol{g}}$ is derived as

$$\underline{\boldsymbol{g}} = \frac{\partial \underline{\boldsymbol{\epsilon}}}{\partial \underline{\boldsymbol{c}}}$$

  - $\underline{\boldsymbol{\epsilon}}$: (2N+1) dimensional paraboloid surface
  - $\underline{\boldsymbol{g}}$: (2N+1) dimensional gradient vector, i.e., $\underline{\boldsymbol{g}} = [\boldsymbol{g}_{-N}, \dots, \boldsymbol{g}_N]$
  - The coefficients of the equalizer are updated by a **step-size** $\mu$ proportional to its magnitude in the direction opposite to its gradient.

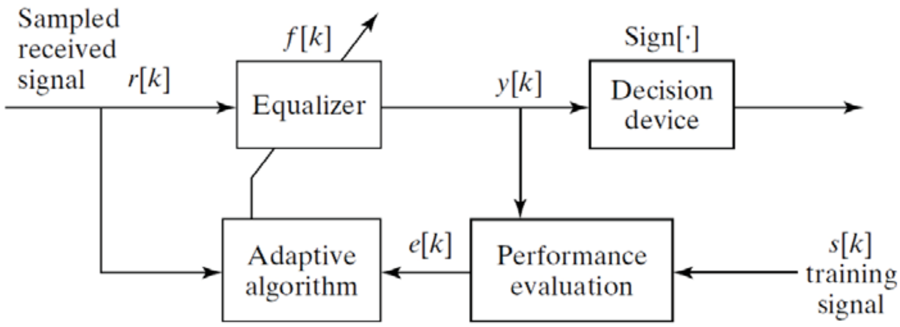
- Therefore, the updating equation can be described as

$$c_{k+1}(n) = c_k(n) - \mu g_k(n), n = -N, \dots, N$$

  - LMS algorithm

# *An Example of Adaptive Equalizer with LMS Algorithm*

- The block diagram of an adaptive equalizer system




  - Equalizer output: $y[k] = \sum_{i=0}^{N_T} f_i r[k-i]$
  - Error estimation: $e[k] = s[k-\delta] - y[k]$, ($\delta$: a delay time)
  - Adaptive algorithm: $f_i[k+1] = f_i[k] + \mu \cdot \{e[k] \cdot r[k-i]\}$
  - The order of equalizer: $N_T$



# *Derivation of Adaptive LMS Equalization*

- The cost function $J_{LMS}$
  - $J_{LMS} = \frac{1}{2} avg\{e^2[k]\} = \frac{1}{N} \sum_{k=0}^{N_T-1} e^2[k]$
  - $e[k] = s[k-\delta] - y[k] = s[k-\delta] - \sum_{i=0}^{n} f_i r[k-i]$
- Use the gradient descent scheme to find $f_i$ and then we have the minimum value of $J_{LMS}$

$$f_i[k+1] = f_i[k] - \mu \left. \frac{\partial J_{LMS}}{\partial f_i} \right|_{f_i = f_i[k]}$$

$$\frac{\partial J_{LMS}}{\partial f_i} = \frac{\partial \left( \frac{1}{2} avg\{e^2[k]\} \right)}{\partial f_i} \approx \frac{1}{2} avg \left\{ \frac{\partial\, e^2[k]}{\partial f_i} \right\} = avg \left\{ e[k] \frac{\partial e[k]}{\partial f_i} \right\}$$

$$\frac{\partial e[k]}{\partial f_i} = \frac{\partial}{\partial f_i} \left\{ s[k-\delta] - \sum_{i=0}^{n} f_i r[k-i] \right\} = -r[k-i]$$

## Derivation of Adaptive LMS Equalization (cont'd)

- The updating of $f_i$
  - $f_i[k+1] = f_i[k] + \mu \cdot avg\{e[k] \cdot r[k-i]\}$
  - Typically, the $avg\{\cdot\}$ is suppressed since the iteration with small step-size $\mu$ has a lowpass (averaging) behavior.
- Finally, the LMS algorithm for direct equalizer coefficient adaption is given as
  - $f_i[k+1] = f_i[k] + \mu \cdot \{e[k] \cdot r[k-i]\}$



## Consideration of Complex Signals

- For QAM modulation, the signals are the complex value.
  - A complex error $e[k] = e_R[k] + je_I[k]$
  - The error power $e[k]e^*[k]$

$$e[k]e^*[k] = e_R^2[k] - je_R[k]e_I[k] + je_R[k]e_I[k] + e_I^2[k]$$
$$= e_R^2[k] + e_I^2[k]$$

  - Optimal equalizer to minimize $\sum_k e[k]e^*[k]$

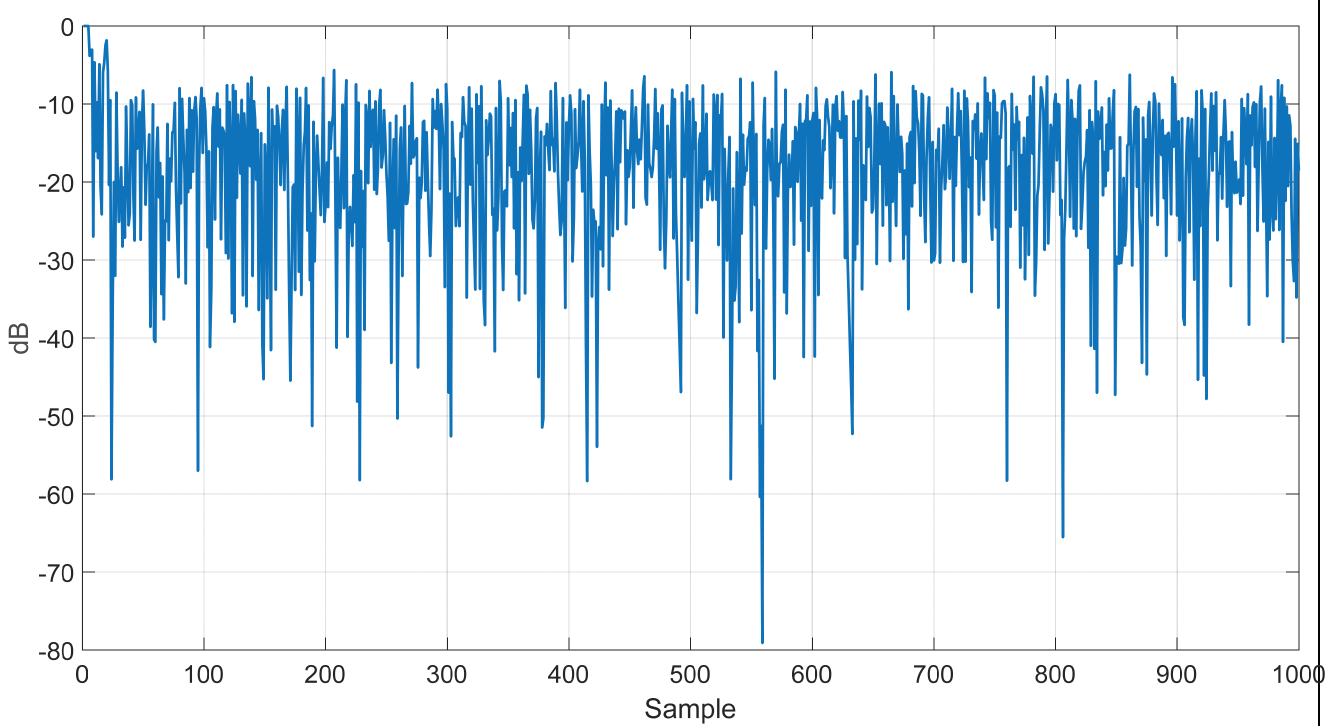
# Design Template for LMS Algorithm

- Channel impulse b=[0.5 1 -0.6];
- Use randn(·) to generate the random data (PAM-2)
- The order of the equalizer: $N_T$=4

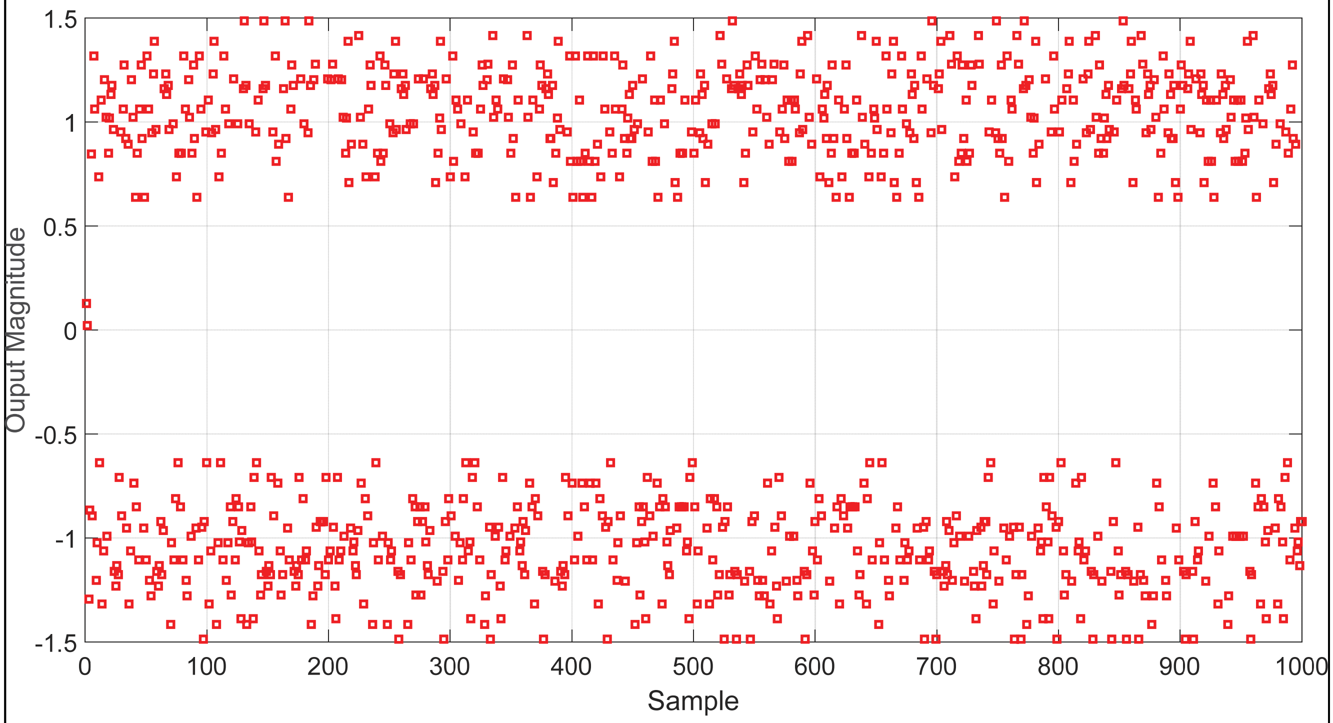
- Check the related impulse responses
  - *b*: the channel impulse response
  - *f* : the equalizer
  - conv(*b*, *f*): the equivalent impulse response including both channel and equalizer



# Error Power of Equalizer

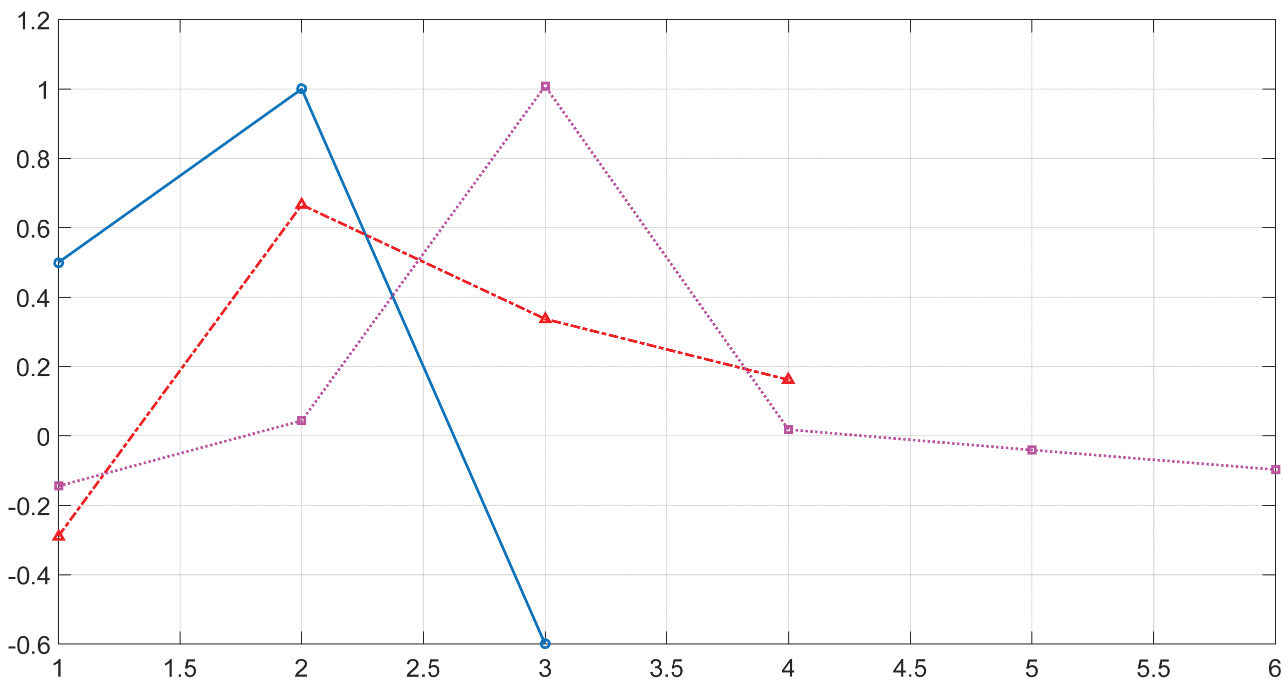


# Output of Equalizer



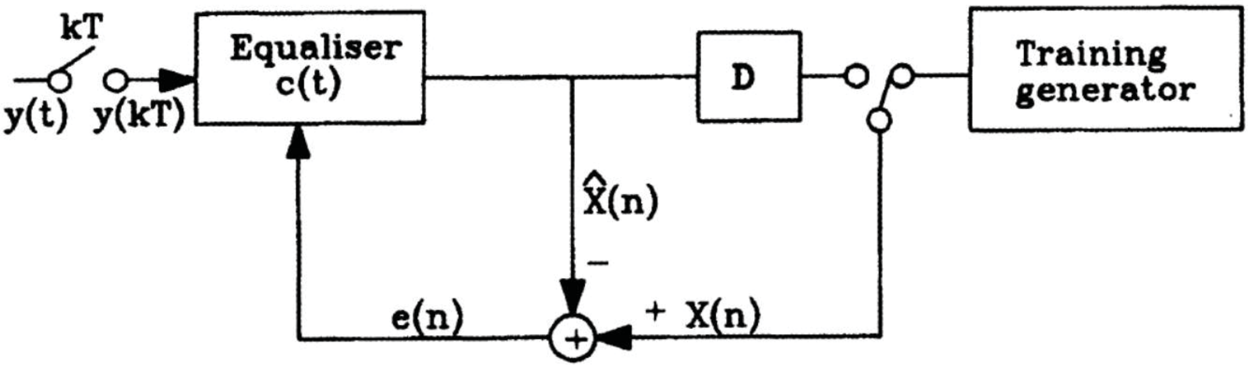



# Coefficients for b, f and conv(b,f)



## *Decision-Directed Adaptive Equalizer*

- Decision directed **automatic adaptive** equalizer
  - **After** a **training** period, the equalizer is required to update its coefficients on the basis of the signal at the output of the **receiver decision device** (slicer), which is not always an error-free.




  - Decision error: $e(n) = x(n) - \hat{x}(n)$
    - Contain both ISI and noise



## *Decision-Directed Adaptive Equalizer (cont'd)*

- The MSE term is used to adjust the coefficients and given as

$$\epsilon = \sum_{n=-\infty}^{\infty} e^2(n)$$

  - Considering the duration of decision-directed, the error signal is described as

$$e(n) = x(n) - \sum_{k=-N}^{N} c(k)y(n-k)$$

  - Then, the MSE is

$$\epsilon = \sum_{n=-\infty}^{\infty} \left[ x(n) - \sum_{k=-N}^{N} c(k)y(n-k) \right]^2$$

# Decision-Directed Adaptive Equalizer (cont'd)

- Minimize the MSE respect to the equalizer coefficients

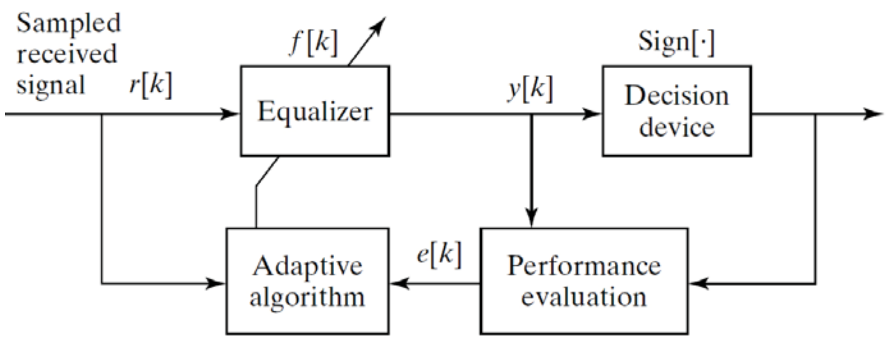
$$\frac{\partial \epsilon}{\partial c(k)} = -2 \sum_{n=-\infty}^{\infty} \left\{ \left[ x(n) - \sum_{k=-N}^{N} c(k) y(n-k) \right] y(n-i) \right\} = 0$$

$$\sum_{n=-\infty}^{\infty} x(n) y(n-i) = \sum_{n=-\infty}^{\infty} \sum_{k=-N}^{N} c(k) y(n-k)\, y(n-i)$$



# An Example of Decision-Directed Linear Equalizer

- The block diagram of an adaptive decision-directed (DD) equalizer system



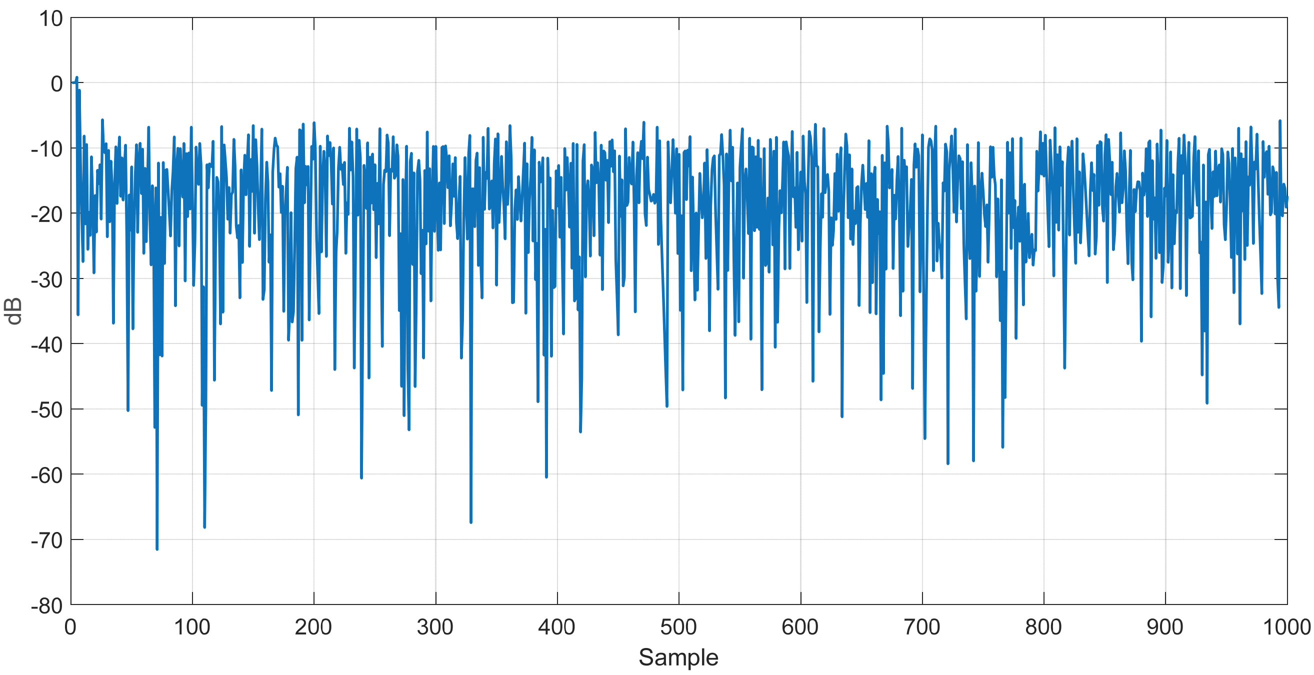

- Decision device for a binary source $\pm 1$
  - The error estimation: $e[k] = sign\{y[k]\} - y[k]$
  - $sign\{y[k]\} = +1$ or $-1$
- The DD LMS algorithm
- $f_i[k+1] = f_i[k] + \mu \cdot \{sign\{y[k]\} - y[k]\} \cdot r[k-i]$

# Design Template for DD LMS Algorim

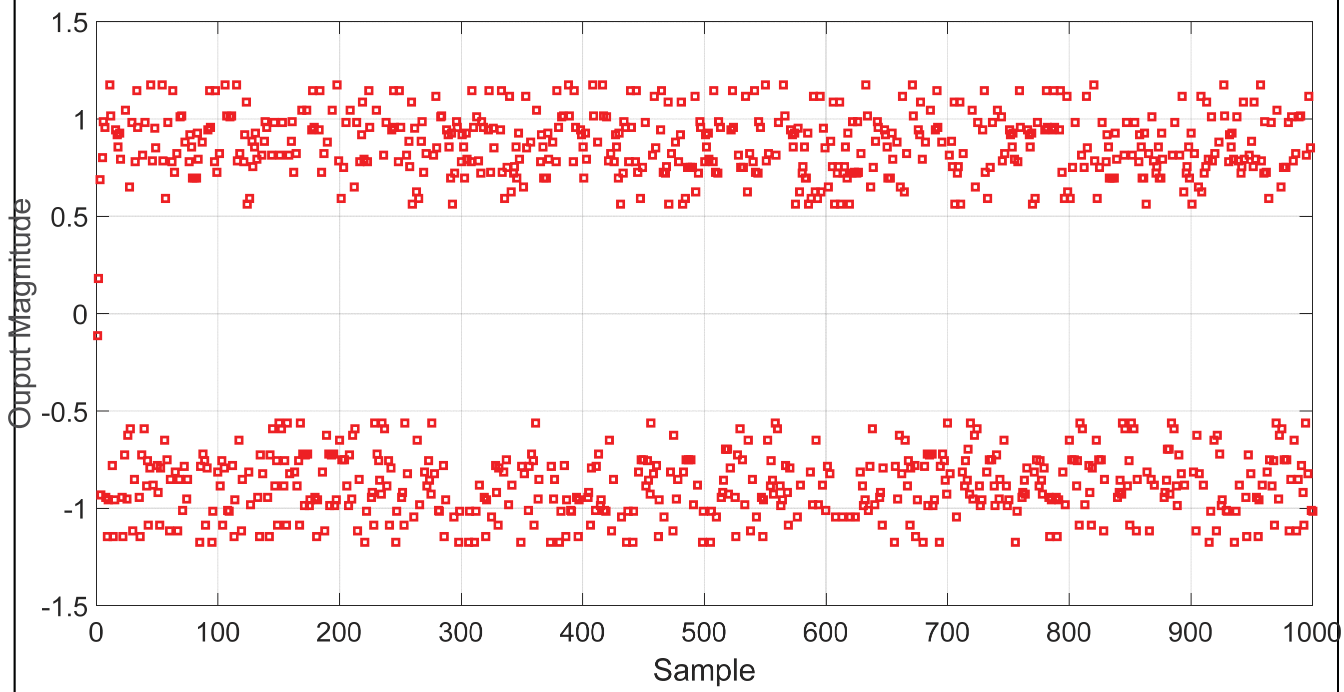
- Channel impulse b=[0.5 1 -0.6];
- Use randn(·) to generate the random data
- The order of the equalizer: $N_T$=4



# Error Power of Equalizer

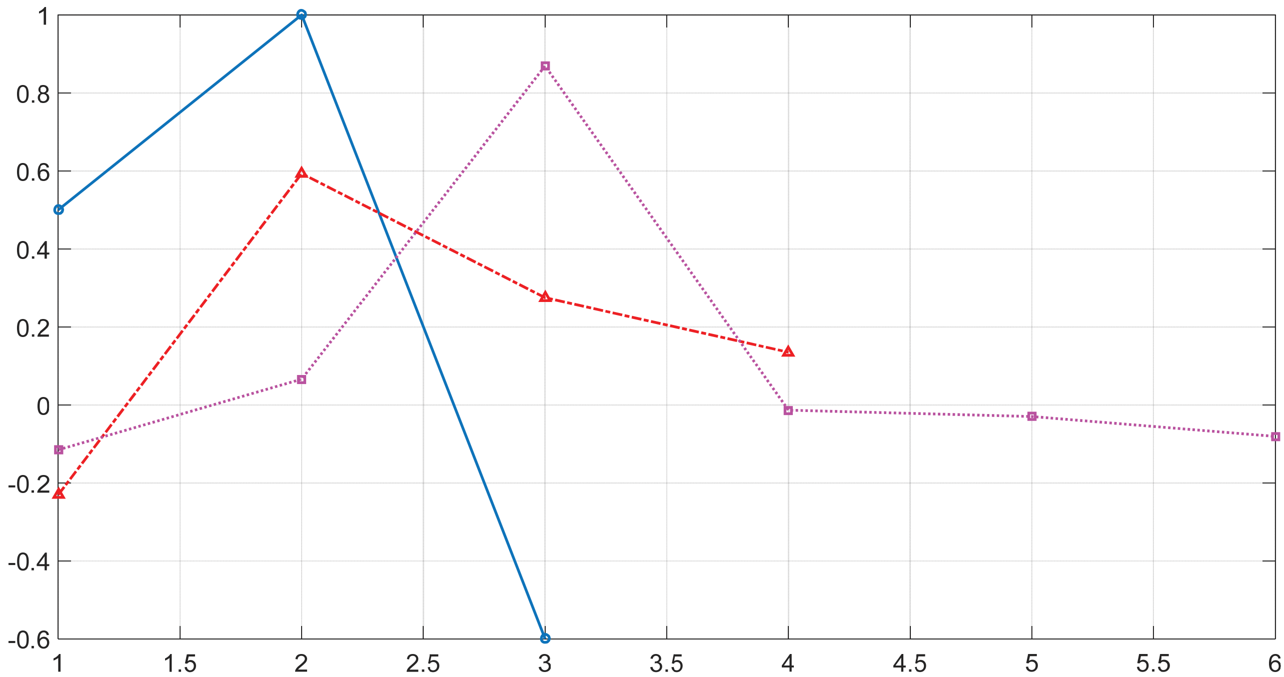


# Output of Equalizer



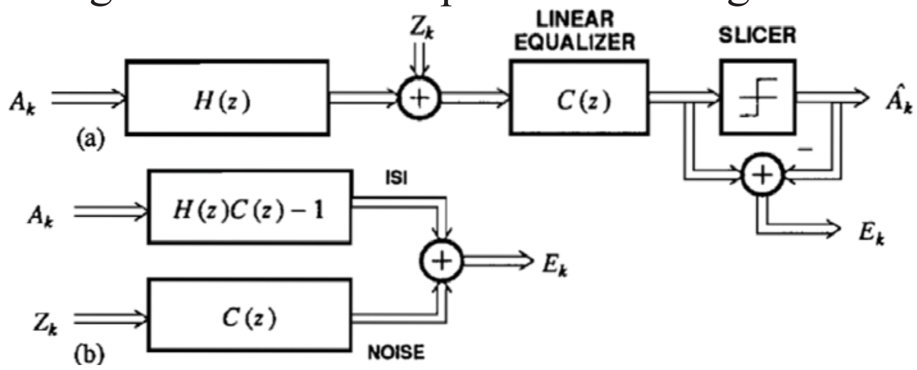



# Coefficients for b, f and conv(b,f)



# A Linear Equalizer Receiver

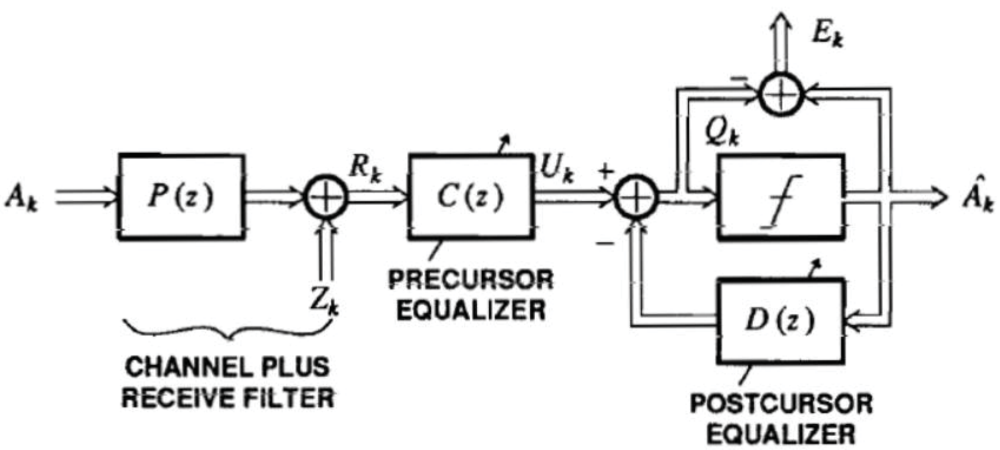
❑ Block diagram of channel equalization using LE






# Decision Feedback Equalizer (DFE)

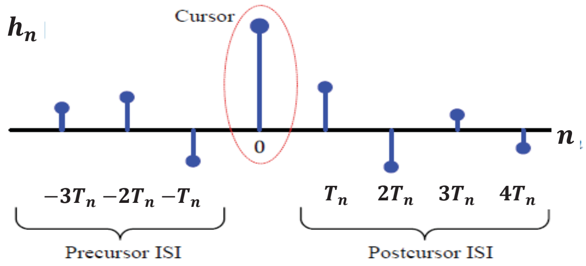
❑ Block diagram of a complete adaptive DFE system




- Precursor equalizer
- Postcursor equalizer

# Decision Feedback Equalizer (cont'd)

❑ Recall pre- and post-cursor

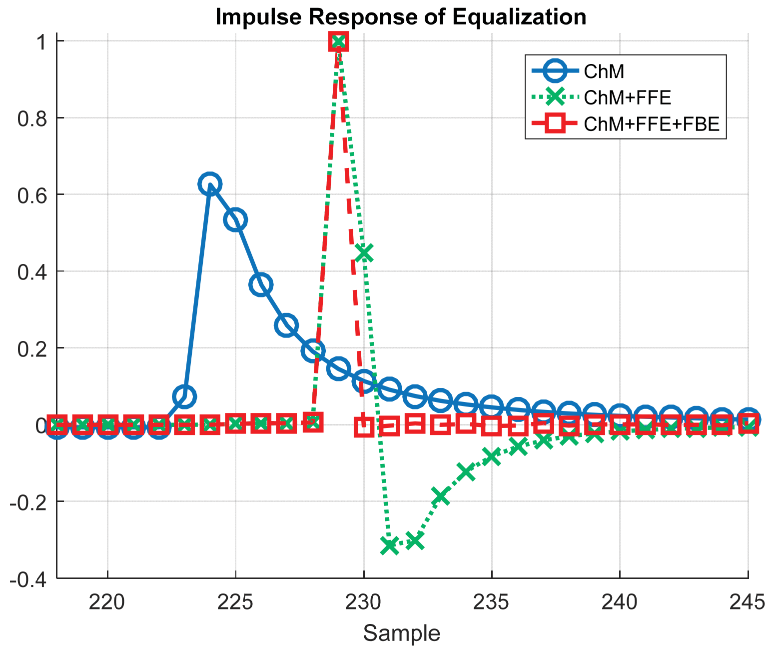



Ref.: https://wirelesspi.com/how-decision-feedback-equalizers-dfe-work/



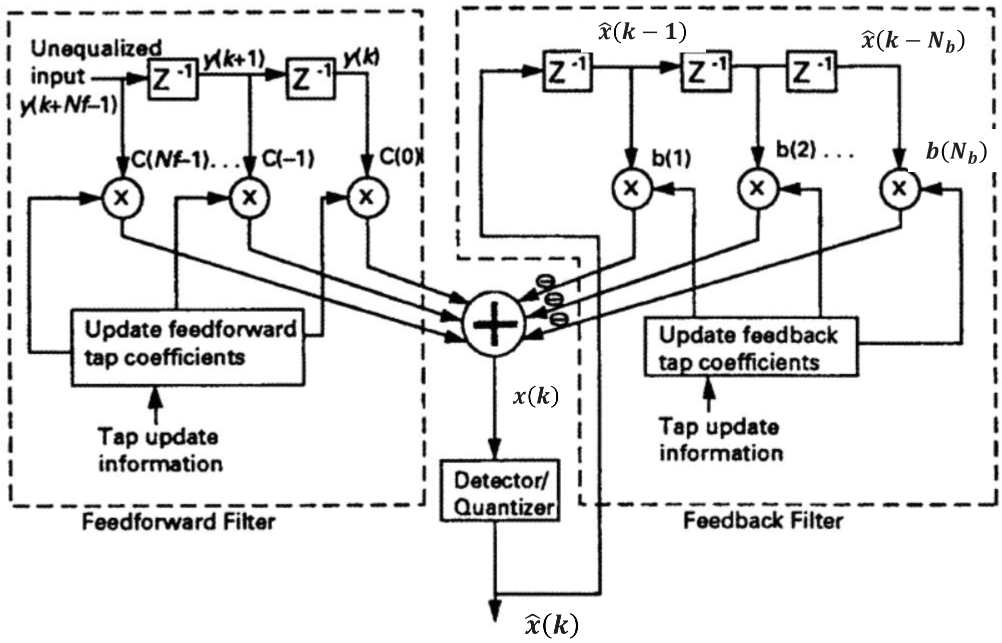
# Decision Feedback Equalizer (cont'd)

❑ Channel model of twisted pair in automotive Ethernet application



# Decision Feedback Equalizer (cont'd)

❑ DFE is composed of a feedforward equalizer and a feedback equalizer.






# Decision Feedback Equalizer (cont'd)

❑ The equalizer signal at the detector input is described as

$$x(k) = \sum_{m=0}^{N_f} c_k(m)y(k+m) - \sum_{p=1}^{N_b} b_k(p)\hat{x}(k-p)$$

- Decision error

$$e(k) = \hat{x}(k) - x(k)$$
$$= \hat{x}(k) - \left\{ \sum_{m=0}^{N_f} c_k(m)y(k+m) - \sum_{p=1}^{N_b} b_k(p)\hat{x}(k-p) \right\}$$

- Minimum MSE criterion

$$\epsilon = \mathbf{E}[|e(k)|^2]$$

# Decision Feedback Equalizer (cont'd)

- Coefficient updating approach
  - FFE:

$$\frac{\partial \epsilon}{\partial c_k} = \frac{\partial}{\partial c_k}\mathbf{E}[|e(k)|^2] = \mathbf{E}\left[2e(k)\frac{\partial e^*(k)}{\partial c_k}\right]$$

$$\frac{\partial e^*(k)}{\partial c_k} = \frac{\partial}{\partial c_k}\left[\hat{x}(k) - \left\{\sum_{m=0}^{N_f} c_k(m)y(k+m) - \sum_{p=1}^{N_b} b_k(p)\hat{x}(k-p)\right\}\right]^*$$

$$= -y^*(k+m)$$

$$\frac{\partial \epsilon}{\partial c_k} = \mathbf{E}[-2e(k)y^*(k+m)]$$

  - Updating:

$$c_{k+1}(m) = c_k(m) + \mu_{\text{FFE}}e(k)y^*(k+m), m = 0, \dots, N_f - 1$$



# Decision Feedback Equalizer (cont'd)

  - FBE:

$$\frac{\partial \epsilon}{\partial b_k} = \frac{\partial}{\partial b_k}\mathbf{E}[|e(k)|^2] = \mathbf{E}\left[2e(k)\frac{\partial e^*(k)}{\partial b_k}\right]$$

$$\frac{\partial e^*(k)}{\partial b_k} = \frac{\partial}{\partial b_k}\left[\hat{x}(k) - \left\{\sum_{m=0}^{N_f} c_k(m)y(k+m) - \sum_{p=1}^{N_b} b_k(p)\hat{x}(k-p)\right\}\right]^*$$

$$= \hat{x}^*(k-p)$$

$$\frac{\partial \epsilon}{\partial b_k} = \mathbf{E}[2e(k)\hat{x}^*(k-p)]$$

  - Updating:

$$b_{k+1}(p) = b_k(p) - \mu_{\text{FBE}}e(k)\hat{x}^*(k-p), p = 0, \dots, N_b - 1$$

- Measured SNR

$$SNR_{measured} = 10 \cdot log_{10} x_k^2 / e_k^2$$

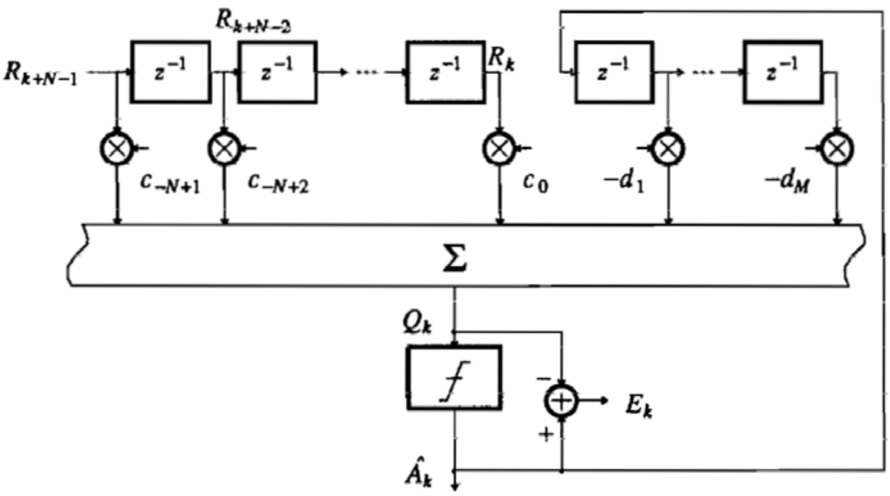
# Architecture of Digital Filter

- Digital filter
  - Finite impulse response (FIR) filter – adaptive equalizer
  - Infinite impulse response (IIR) filter
- FIR filter (b) (transversal filter, tapped-delay-line, direct-form)
  - $y_n = \sum_{k=0}^{N-1} h_k \cdot x_{n-k}$
  - $h = h_0 + h_1 z^{-1} + \cdots + h_{N-1} z^{-(N-1)}$
  - $N$-tap
  - $N$ coefficients (ROMs)
  - $N$ storages ($x_n$)
  - $N$ multiplications
  - $N$-$1$ additions



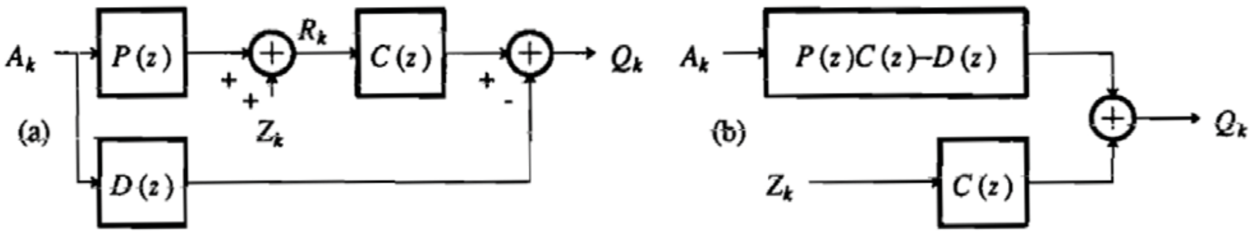
# Adaptive DFE

- Block diagram



# Adaptive DFE (cont'd)

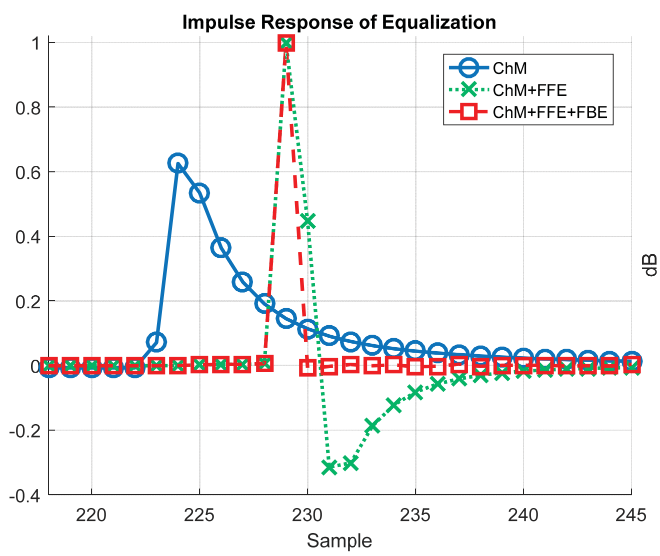
- ❑ Equivalent model of channel and equalization

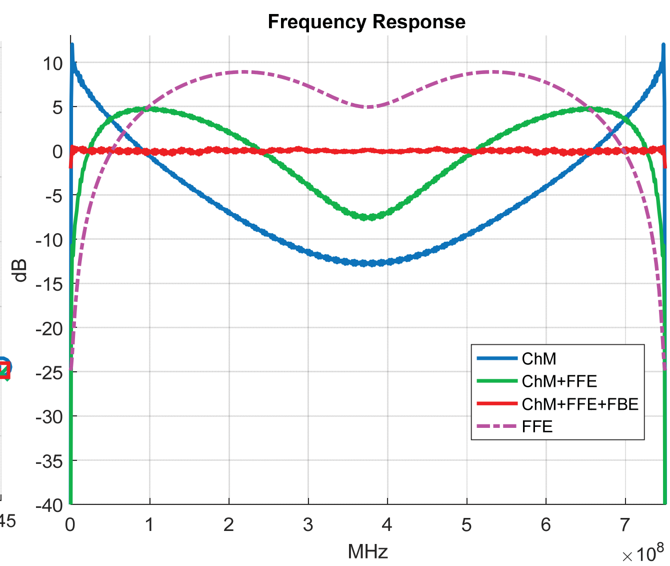



  - ■ Supper-position theorem



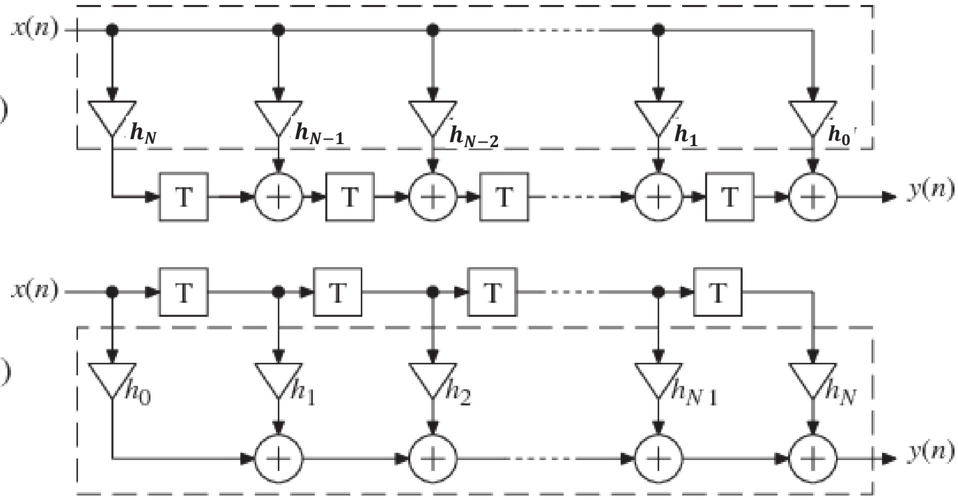
# Case Study for Automotive Ethernet PHY



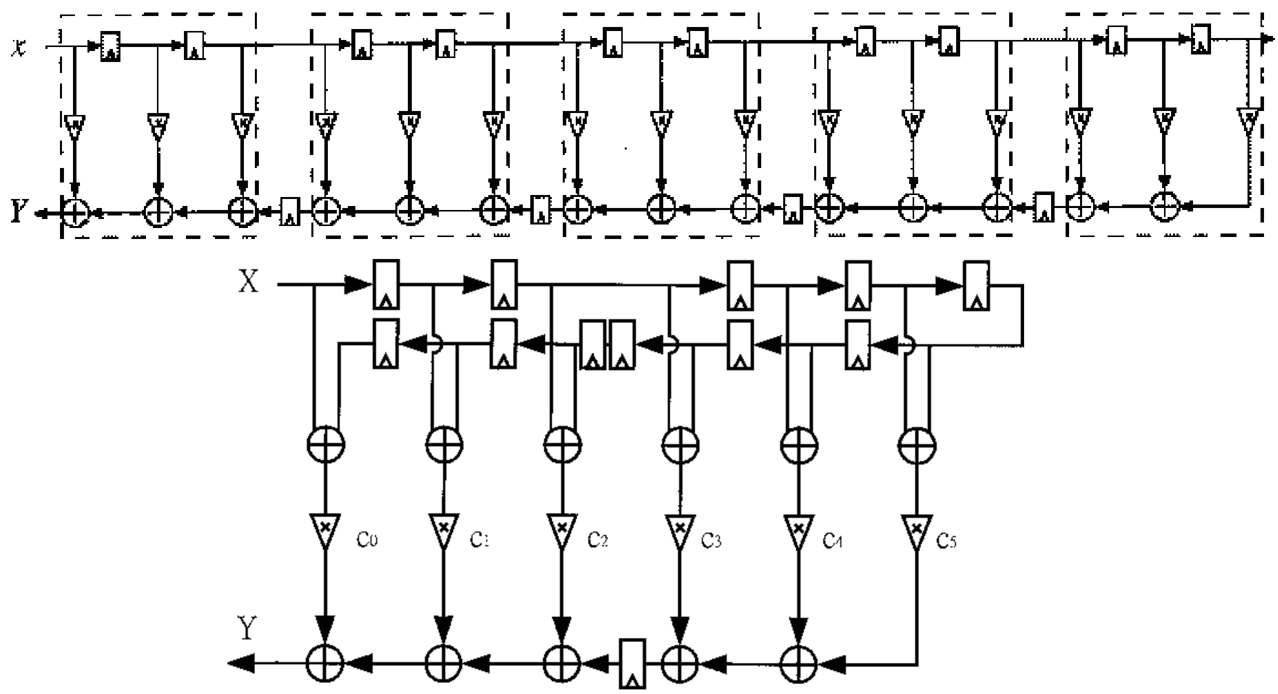
# Architecture of Digital Filter (cont'd)

- Transposed form (a)
  - High-speed operation (intrinsically, pipelined architecture)
  - More hardware cost on the storages and the additions



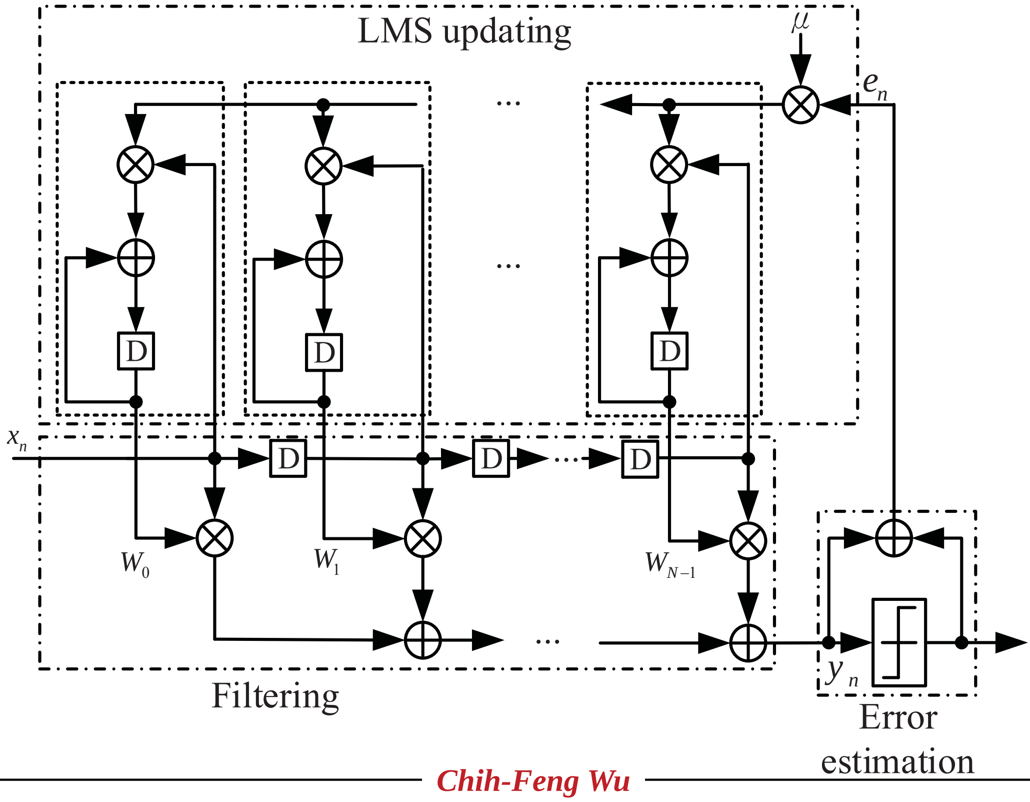



# Alternative Architecture of FIR Filter




Ref.: IEEE TCAS-II, 2004

# Block Diagram of Linear Equalization






# Various LMS Algorithms

- Filtering: $y_n = \sum_{k=0}^{N-1} f_k \cdot x_{n-k}$
- Decision error: $e_n = \hat{x}_n - y_n$; $\hat{x}_n = Q[y_n]$, $Q[\cdot]$: slicer
- Sign LMS Algorithm

$$f_{n+1}[k] = f_n[k] + \mu \cdot \mathbf{sign}(e_n) \cdot x_{n-k}$$

- Sign-Sign LMS Algorithm

$$f_{n+1}[k] = f_n[k] + \mu \cdot \mathbf{sign}(e_n) \cdot \mathbf{sign}(x_{n-k})$$

- Delayed LMS Algorithm

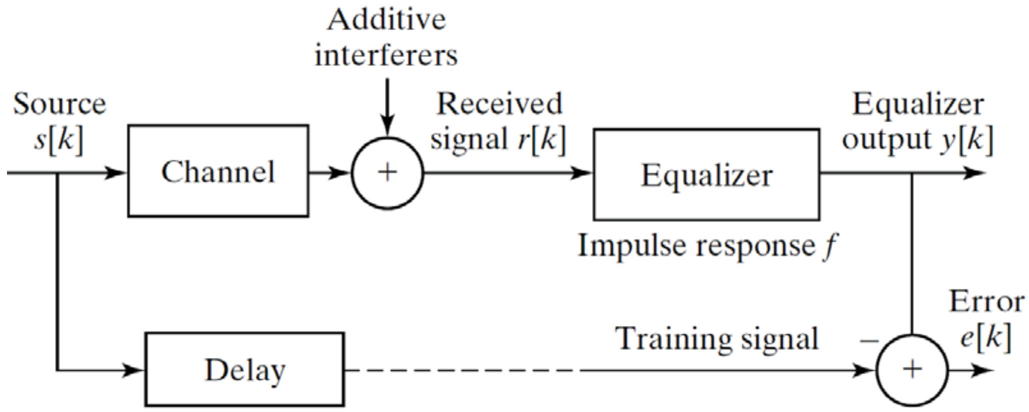
$$f_{n+1}[k] = f_n[k] + \mu \cdot e_{n-\mathrm{D}} \cdot x_{n-k-\mathrm{D}}$$

- Sign-Delayed LMS

$$f_{n+1}[k] = f_n[k] + \mu \cdot \mathbf{sign}(e_{n-\mathrm{D}}) \cdot x_{n-k-\mathrm{D}}$$

## *Adaptive Linear Equalization*

- A training sequence available to build (or train) an equalizer
- The problem of the linear equalization




  - $s[k]$: a prearranged training sequence
  - Equalizer: an FIR filter to find $y[k] \approx s[k-\Delta]$ for some specific $\Delta$



## *Equalization Approach*

- FFE and FBE filtering

$$\overline{w}_n = \sum_{k=0}^{N_f-1} f_k \cdot w_{n-k}, \quad \overline{z}_n = \sum_{k=0}^{N_b-1} b_k \cdot \hat{x}_{n-k}, \quad y_n = \overline{w}_n - \overline{z}_n$$

- Error estimation
  - Blind equalization: $e_n = (y_n^2 - \gamma) \cdot y_n$ (Constant-Modulus Algorithm)
  - Decision-directed: $e_n = y_n - \hat{x}_n$
- Weight updating based on LMS algorithm

$$f_k(n+1) = f_k(n) - \mu_{FFE} \cdot w_n \cdot e_n$$

$$b_k(n+1) = b_k(n) + \mu_{FBE} \cdot \hat{x}_n \cdot e_n$$

- Measured SNR

$$\mathrm{SNR}_{measured} = 10 \cdot \log_{10} y_n^2 / e_n^2$$